



Technical University
of Denmark

Phase-Enhanced Filter Bank CCA for Single-Channel SSVEP BCI

22053 Principles of Brain Computer Interface

Authors of Group 2

Student ID	First name	Last name
s251976	Leonel	Vázquez Carrasco
s204267	Julie	Verne Henriksen

January 20, 2026

Contents

1	Abstract	iii
2	Introduction	1
2.1	Related Work	2
2.2	Motivation	3
3	Methods	4
3.1	Dataset and Pre-processing	4
3.1.1	Benchmark SSVEP Dataset	4
3.1.2	Data Acquisition and Structure	4
3.1.3	Offline Pre-processing and Channel Selection	5
3.2	Proposed Pipeline: Single-Channel FBCCA with Phase Feature Integration	6
3.2.1	Filter Bank Canonical Correlation Analysis (FBCCA) Feature Extraction .	7
3.2.2	Phase Feature Derivation	10
3.2.3	Classification Strategy	11
3.3	Experimental Design	11
3.4	Performance Metrics and Statistical Analysis	12
3.4.1	Primary Performance Metrics	12
3.4.2	Descriptive and Comparative Statistics	13
3.4.3	Analysis of Data Length Effect	13
4	Results	14
4.1	Single-channel Performance	14
4.1.1	Effect of Time-Window Length	15
4.2	9-channel Performance	16
4.2.1	Effect of Time-Window Length	17

5 Discussion	18
5.1 Related Work	19
5.2 Learning Values	19
5.3 Future Work	20
6 Conclusion	21
References	I
Appendices	III
A Statistical Test Results	III
B Python code	X

1 Abstract

Purpose: Steady state visually evoked potential (SSVEP) based brain computer interfaces (BCI) offer high signal to noise ratio and information transfer rates (ITR) with minimal user training, but usually rely on multichannel EEG recordings to achieve robust and accurate performance. These systems increase hardware complexity, setup time, and user burden, which limits real world applicability and availability. This work investigates whether adding explicit phase information inherent in joint frequency and phase modulated stimuli can improve decoding performance in a single channel SSVEP system.

Materials and methods: Using a public benchmark dataset with 40 stimulation targets, an adapted single channel Filter Bank Canonical Correlation Analysis (FBCCA) pipeline was combined with a sequential phase feature extraction module. Phase was estimated for only the most likely frequency candidates identified by FBCCA and encoded using sine and cosine values before classification with a nonlinear support vector machine. Performance was evaluated using leave one subject out cross validation and compared against single channel and nine channel FBCCA baselines.

Results: Incorporating phase features improves single channel performance across subject and channels. For the best performing channel, Oz, mean classification accuracy increased from 62.30% to 65.38% and mean ITR increase from 58.9 bits/min to 63.91 bits/min using a 2 second analysis window. The largest relative gains were observed at shorter time windows around 1 second, where accuracy improved by 8 percentage points and ITR improved by 14.52 bits/min. For the 9 channel configuration, however, meaningful improvements were only made around the smaller time windows with peak performance increase at 0.75 s with mean accuracy increase by 3.56 percentage points and ITR by 9.16 bits/min.

Conclusion: These findings show that explicit phase information is a complementary feature for single channel SSVEP decoding. The proposed approach improves performance without increasing hardware complexity, supporting the development of more practical and accessible BCI systems.

2 Introduction

Brain computer interfaces (BCIs) enable direct communication between neural activity and external devices or applications by translating brain signals into control commands. The primary motivation and use for the development of such systems lies in assisting communication and control for individuals with motor impairments, such as locked in syndrome, spinal cord injuries, or advanced neurodegenerative diseases, where conventional neuromuscular pathways and biomechanical function is unavailable [6, 13].

BCI modalities for signal acquisition include many non-invasive options, where electroencephalography (EEG) remains the most widely used, due to its safety, portability, low cost, and high temporal resolution. EEG-based BCIs can therefore exploit real-time neural signals that are elicited and detected, including motor imagery (MI), event related potentials (ERP), and steady state visually evoked potentials (SSVEP). Each of these BCI paradigms have advantages and disadvantages related to user training, robustness, accuracy, information transfer rate (ITR), and overall system complexity, which makes signal processing and decoding strategies for user intent central to system performance and usability [6, 14].

SSVEP-based BCI systems have emerged as one of the most effective paradigms for high speed communication. When the user fixates on a periodically flickering visual stimulus, responses in the visual cortex oscillate at the stimulus frequency and its harmonics, eliciting a high signal-to-noise ratio (SNR) result with minimal user training. This property makes an SSVEP-based systems well suited for spellers and rapid selection tasks, where the frequency recognition is important [13, 6]. A typical SSVEP system consists of visual stimulus, EEG signal acquisition, signal pre-processing, feature extraction and classification of user intent to translate to a command signal to the BCI application in question. An overview of a BCI system setup is shown in Fig. 1. Here, system performance depends on signal processing choices, including the number

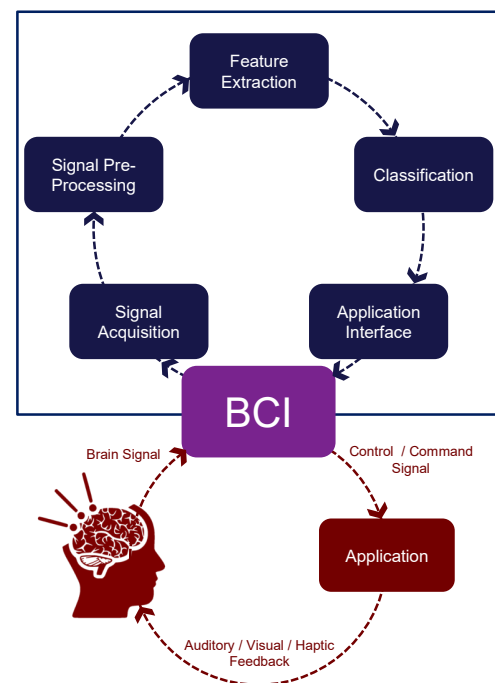


Figure 1: Illustration of the basic BCI components. Adapted from 22053: Principles of Brain Computer Interfaces Lecture 3 on Signal Processing in BCIs [7].

of EEG channels, noise removal and feature choices to ensure correct decoding of user intent [14].

High performing SSVEP systems have relied on multichannel EEG recordings over the occipital and parietal regions of the brain to enhance spatial resolution and improve robustness across subjects and recording settings. Multichannel systems, however, also increase setup time, user burden, hardware complexity and system costs, limiting the system availability and application in real world settings.

2.1 Related Work

Recent work on SSVEP-based BCI systems has emphasized low-complexity and accessible system designs, particularly through the use of single-channel EEG systems. Several studies have demonstrated that single-channel SSVEP systems can achieve competitive accuracy and ITR when appropriate signal processing techniques are employed. Autthasan et al did in 2020 show that frequency recognition and amplitude modulation of SSVEPs can reliably use consumer-grade, single-channel EEG devices, showcasing the feasibility of low-cost and portable systems [2]. Karunasena et al demonstrated in 2021 that frequency-domain Euclidean distance measures applied to single-channel EEG signal can effectively discriminate SSVEP targets [5]. More recently, Preetha et al presented in 2025 a real-time, wireless, single-channel SSVEP speller that achieves high accuracy and ITR using a modified power spectral density (PSD) based decoding pipeline. These works argue that single-channel SSVEP systems can be viable solutions for practical and available real-world BCI solutions [9].

Significant progress has been made in improving SSVEP decoding performance through correlation-based methods. Canonical correlation analysis (CCA) and its many modified approaches have become a standard method for SSVEP frequency recognition, and Chen et al introduced in 2015 a filterbank canonical correlation analysis (FBCCA) to utilise harmonic components of the SSVEP response, resulting in significant performance gains compared to standard CCA [4]. While FBCCA was originally evaluated for multichannel EEG recordings, its filterbank design and harmonic weights provides insights and high-dimensional frequency information relevant for reduced-channel systems. Further extensions combining filter bank approaches with supervised CCA variants, such as multiway CCA (MwayCCA), have been shown to improve performance with a 9-channel EEG system [3].

A widely used benchmark dataset for evaluating SSVEP decoding methods is the benchmark dataset introduced by Wang et al in 2017 [12]. This dataset employs a joint frequency and phase modulation scheme in a high-speed SSVEP speller with a dense set of 40 visual targets, enabling comparison of stimulus decoding and target identification. While the dataset is commonly used for benchmarking multichannel decoding techniques, the stimulation design also provides an op-

portunity to investigate the phase-related information that is not explicitly exploited in the standard FBCCA-based approaches. Sözer et al utilised this dataset in 2018 to show that incorporating phase information alongside subject-specific frequency-based features improves single-channel SSVEP decoding accuracy compared to PSD-based methods [11].

2.2 Motivation

The motivation of this work is to improve the availability and practicality of SSVEP-based BCI systems by investigating accuracy improvements in the simplest possible system configuration, a single-channel EEG. Reducing channel counts lowers hardware cost, setup time, and user burden, which are critical factors for real-world and assistive BCI applications. However, as single-channel systems lack spatial information, placing greater emphasis on efficient signal processing and feature extraction is needed to achieve reliable performance.

Building on prior work demonstrating that single-channel SSVEP decoding is possible, this work aims to enhance frequency classification accuracy by incorporating phase information as additional validation features. While FBCCA effectively uses the harmonic frequency components of the elicited stimulus response, it does not directly assess phase consistency across its target predictions. The joint frequency and phase modulation employed in the benchmark dataset offers an additional signal domain that we can use for this purpose.

By comparing a FBCCA-based frequency recognition approach with an extended method that incorporates phase validation features, this work investigates whether phase information can improve classification accuracy in a single-channel setting. The overall goal is to demonstrate that algorithmic adjustments can achieve a meaningful performance increase and support the development and usage of low-complexity, more accessible SSVEP-based BCI systems.

3 Methods

3.1 Dataset and Pre-processing

3.1.1 Benchmark SSVEP Dataset

This study utilizes the publicly available benchmark dataset for steady-state visual evoked potential (SSVEP)-based brain-computer interfaces [12]. This dataset was specifically designed to facilitate the comparison of stimulus coding and target identification methods. It comprises electroencephalogram (EEG) recordings from 35 healthy subjects (8 experienced, 27 naive) performing a cued target selection task with a 40-target visual speller. The speller’s stimuli were encoded using a Joint Frequency and Phase Modulation (JFPM) approach, where each target k is associated with a unique frequency f_k and phase θ_k :

$$f_k = 8.0 + 0.2 \cdot (k - 1) \text{ Hz}, \quad k = 1, \dots, 40, \quad (1)$$

$$\theta_k = 0 + 0.5\pi \cdot (k - 1) \text{ rad}. \quad (2)$$

This results in frequencies ranging from 8.0 to 15.8 Hz. The dataset provides precise synchronization between the stimulus events and the EEG recordings, which is crucial for phase-locked analysis.

3.1.2 Data Acquisition and Structure

During the experiment, 64-channel EEG data were recorded at 1000 Hz using a Synamps2 system (Neuroscan, Inc.) with a bandpass filter (0.15 - 200 Hz). The reference electrode was placed at the vertex (Cz). For each subject, the experiment consisted of 6 blocks, each containing 40 trials (one per target) presented in random order. Each trial lasted 6 seconds: a 0.5 s visual cue, a 5 s stimulation period where all targets flickered simultaneously, and a final 0.5 s blank screen [12].

The provided data files contain pre-processed epochs. The continuous data were segmented into 6-second epochs (from 0.5 s pre-cue to 0.5 s post-stimulus) and downsampled to 250 Hz. Consequently, each subject’s data is stored in a 4-dimensional matrix of dimensions $[64, 1500, 40, 6]$, corresponding to [channels, time points, targets, blocks].

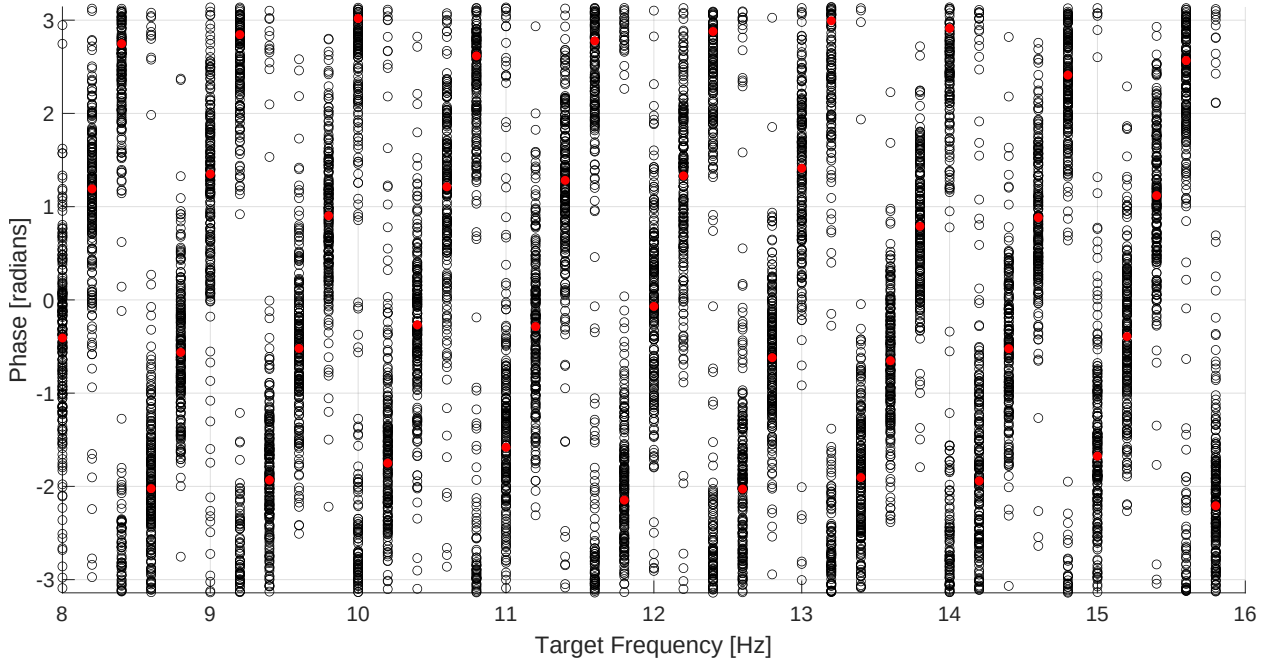


Figure 2: Empirical demonstration of phase discriminability in the JFPM-coded SSVEP dataset. For each of the 40 stimulation frequencies (8.0–15.8 Hz), the estimated single-trial SSVEP phase is plotted (black dots) for all subjects and trials, extracted from the Oz channel. The circular mean phase for each frequency, calculated via von Mises statistics, is overlaid (red circles).

3.1.3 Offline Pre-processing and Channel Selection

The primary offline pre-processing step applied in this work was a software-based 50 Hz notch filter, implemented using an infinite impulse response (IIR) comb filter to suppress any residual powerline interference. The filtering was applied in a zero-phase manner using `filtfilt` to avoid distorting the temporal structure of the SSVEPs, which is critical for subsequent phase analysis.

For the multi-channel baseline analysis, we selected the set of nine parietal-occipital electrodes identified as optimal for SSVEP detection in the original FBCCA study [4]: Pz, PO5, PO3, POz, PO4, PO6, O1, Oz, and O2. This selection provides a robust spatial filter over the visual cortex. The performance using this 9-channel set serves as a benchmark against which our proposed single-channel approach is evaluated. The single-channel analysis involved systematically testing the performance of each of these nine individual channels to identify the most informative recording site.

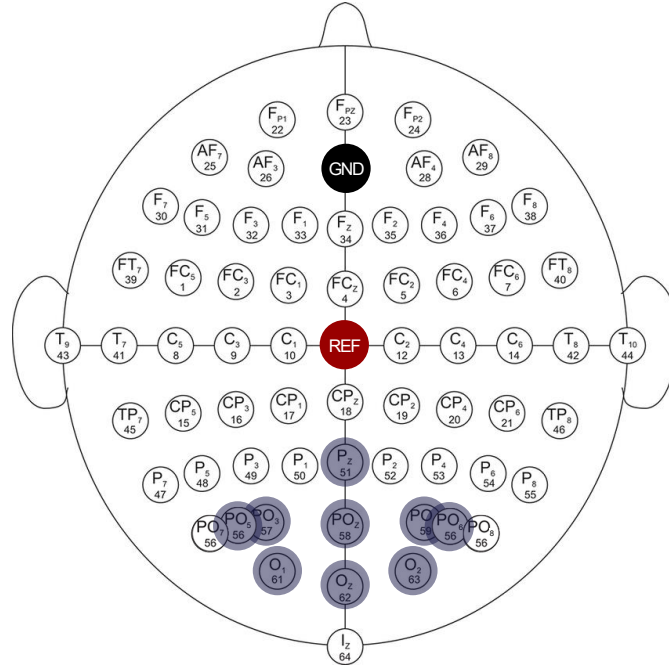


Figure 3: Illustration of the choice electrodes recommended for use in the original FBCCA study, as well as in the Benchmark dataset [4, 12]. Electrode layout is adapted from Qiu et al. [8].

3.2 Proposed Pipeline: Single-Channel FBCCA with Phase Feature Integration

The core contribution of this work is an integrated processing pipeline designed to maximize the information extracted from a single EEG channel for SSVEP-based target identification. The pipeline, illustrated in Fig. 4, synergistically combines the robust harmonic analysis of Filter Bank CCA with a novel cascaded phase estimation module, specifically designed to exploit the Joint Frequency and Phase Modulation (JFPM) of the stimuli.

The pipeline operates as follows for a single trial of single-channel EEG data:

1. **Pre-processing & Epoch Extraction:** The raw EEG signal from the selected channel is notch-filtered at 50 Hz. A 2.0 second data epoch, time-locked to the stimulus onset and shifted by a 140 ms visual latency, is extracted for analysis (Sec. 3.2.1.1).
2. **FBCCA Feature Extraction:** The epoch is processed through the adapted single-channel FBCCA module (Sec. 3.2.1). This involves decomposition via a 7-sub-band filter bank, followed by canonical correlation analysis between each sub-band component and sinusoidal reference signals for all 40 stimulus frequencies. The sub-band correlations are aggregated using an exponential weighting scheme, producing a 40-dimensional feature vector $\tilde{\rho}$ representing the evidence for each possible stimulus frequency.

3. **Candidate Selection & Phase Feature Extraction:** The top three candidate frequencies are identified from $\tilde{\rho}$. For each candidate, a high-resolution FFT is computed on the original epoch. The phase at each candidate frequency is estimated by locating the spectral peak within a narrow band, and the circular phase value is encoded into its sine and cosine components, yielding six linear features (Sec. 3.2.2).
4. **Feature Integration & Classification:** The 40 FBCCA features and the 6 phase features are concatenated into a unified 46-dimensional feature vector. This vector is standardized and fed into a non-linear Support Vector Machine (SVM) classifier with an RBF kernel, which is trained in a subject-independent manner using a Leave-One-Subject-Out protocol (Sec. 3.2.3).

The novelty of this pipeline lies in two key aspects: (1) the successful adaptation of the multi-channel FBCCA method to a single-channel input, maintaining its ability to capture harmonic information, and (2) the cascaded phase feature extraction that leverages the initial FBCCA frequency estimate to guide robust phase estimation, thereby adding a complementary, discriminative information stream derived from the JFPM coding.

3.2.1 Filter Bank Canonical Correlation Analysis (FBCCA) Feature Extraction

The frequency and harmonic information from the SSVEPs was extracted using the Filter Bank Canonical Correlation Analysis (FBCCA) method, following the implementation detailed in [4]. The core innovation of our adaptation lies in applying this powerful method to a single EEG channel, moving towards a more practical BCI setup.

3.2.1.1 Epoch Selection

For each trial, a data epoch of 2.0 seconds was extracted for analysis. To account for the physiological delay in the visual pathway, a latency shift of 140 ms was applied relative to the stimulus onset [4, 12]. Consequently, the analysis window spanned from $t = 0.64$ s to $t = 2.64$ s post-stimulus onset, capturing the steady-state response.

3.2.1.2 Filter Bank Structure

The FBCCA method decomposes the single-channel SSVEP signal $\mathbf{x}$ into $N = 7$ sub-band components $\mathbf{x}_{\text{SB}_n}$ using a bank of zero-phase Chebyshev Type I band-pass filters. The filter bank was designed according to the M_3 scheme from [4], where the n -th sub-band ($n = 1, \dots, 7$) has a passband from $n \times 8$ Hz to an upper bound of 90 Hz. Transition bands of 2 Hz (low) and 10 Hz (high) were used, with a passband ripple of 0.5 dB and a stopband attenuation of 40 dB. This design ensures comprehensive coverage of the fundamental and harmonic SSVEP components.

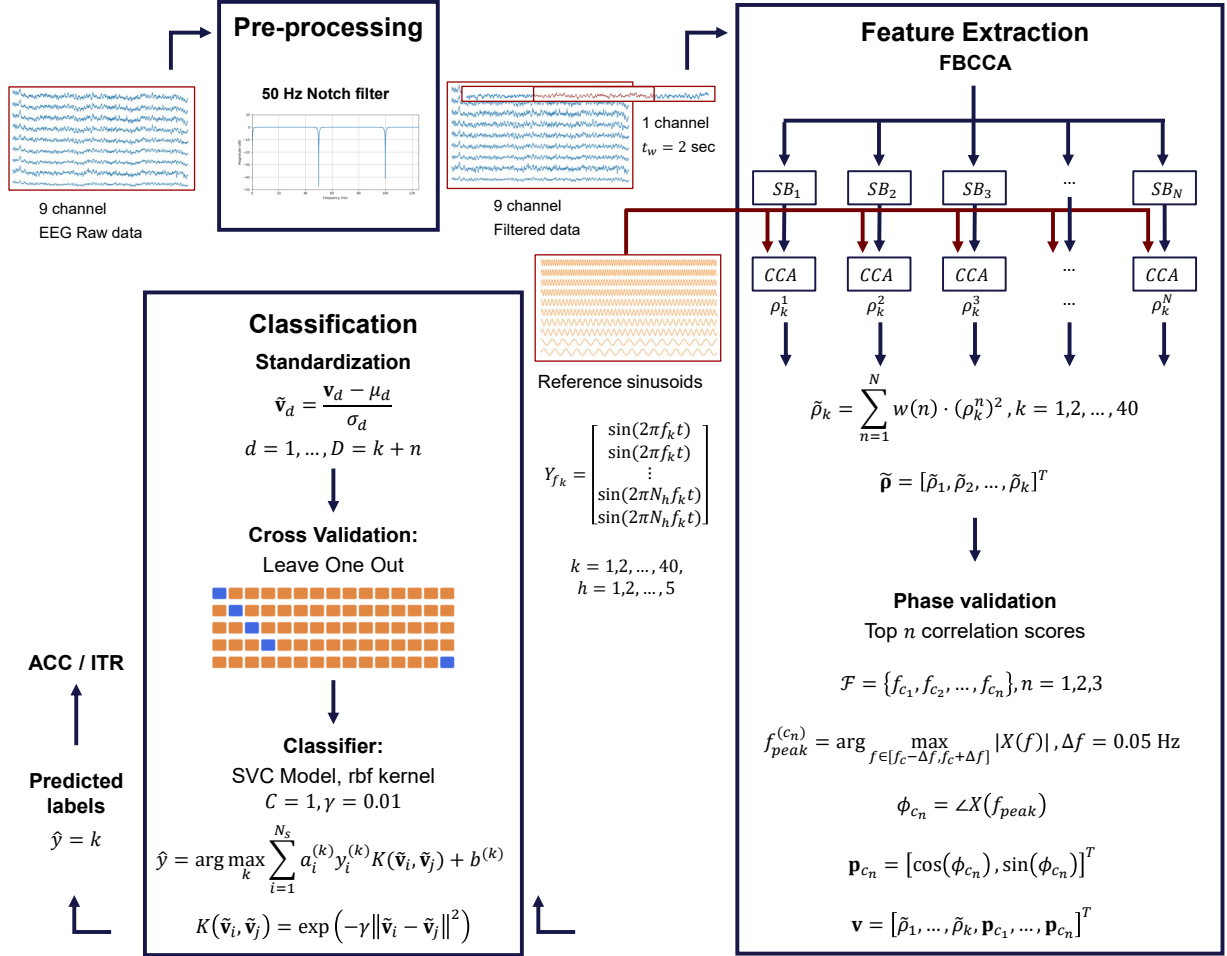


Figure 4: Pipeline of the Phase-validated FBCCA-based SSVEP classifier. FBCCA system components are drawn from Chen et al, 2015 [4].

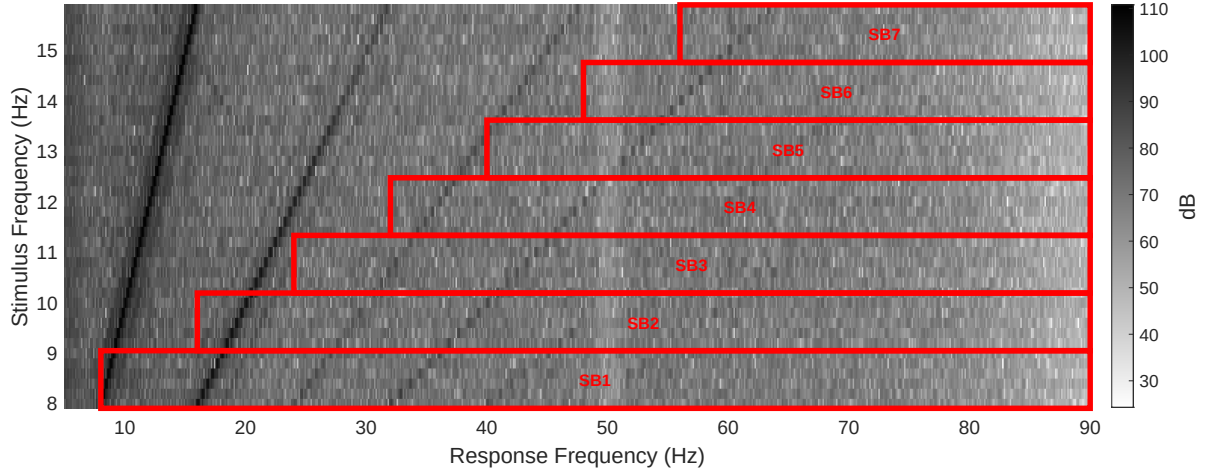


Figure 5: Filter bank design (M3 method) and empirical SSVEP harmonic response. The background image depicts the signal-to-noise ratio (SNR, in dB) as a function of stimulus frequency (y-axis) and EEG response frequency (x-axis), averaged across all subjects and trials for the Oz channel. Superimposed in red are the seven sub-bands of the M_3 filter bank used in the adapted FBCCA method [4].

3.2.1.3 Canonical Correlation Analysis per Sub-band

Within each sub-band $\mathbf{x}_{SB_n}$, a canonical correlation analysis (CCA) is performed. CCA finds the linear combination that maximizes the correlation between the filtered EEG signal and a set of artificial reference signals $\mathbf{Y}_{f_k}$. For each candidate stimulus frequency f_k ($k = 1, \dots, 40$), the reference signals are constructed as sinusoidal templates containing the fundamental and $N_h = 5$ harmonics:

$$\mathbf{Y}_{f_k} = \begin{bmatrix} \sin(2\pi f_k t) \\ \cos(2\pi f_k t) \\ \vdots \\ \sin(2\pi N_h f_k t) \\ \cos(2\pi N_h f_k t) \end{bmatrix} \quad (3)$$

The maximum canonical correlation coefficient ρ_k^n between $\mathbf{x}_{SB_n}$ and $\mathbf{Y}_{f_k}$ is computed for each frequency f_k . The correlation coefficients from all sub-bands are aggregated into a single discriminative feature for each stimulus frequency. Following [4], a weighted sum of squares is computed:

$$\tilde{\rho}_k = \sum_{n=1}^N w(n) \cdot (\rho_k^n)^2, \quad (4)$$

where the weight $w(n)$ for the n -th sub-band follows an exponential decay to emphasize lower, higher SNR harmonics: $w(n) = n^{-a} + b$, with $a = 1.25$ and $b = 0.25$. This yields a 40-dimensional feature vector $\tilde{\boldsymbol{\rho}} = [\tilde{\rho}_1, \tilde{\rho}_2, \dots, \tilde{\rho}_{40}]^T$ for each trial, where a larger $\tilde{\rho}_k$ indicates a stronger evidence for the corresponding stimulus frequency f_k .

3.2.2 Phase Feature Derivation

To leverage the explicit phase information encoded in the JFPM stimuli, a cascaded phase feature extraction module was implemented. This module operates on the output of the FBCCA stage, ensuring that phase estimation is performed only for the most probable stimulus frequencies, thereby increasing robustness.

The 40-dimensional FBCCA feature vector $\tilde{\rho}$ is first used to identify the most likely target frequencies. The indices corresponding to the $n = 3$ highest values in $\tilde{\rho}$ are selected, forming a set of candidate frequencies $\mathcal{F}_{\text{cand}} = \{f_{c1}, f_{c2}, \dots, f_{cn}\}$. This step focuses subsequent phase analysis, reducing the influence of noise from irrelevant frequency bands.

For each candidate frequency $f_c \in \mathcal{F}_{\text{cand}}$, the associated SSVEP phase ϕ_c is estimated from the same 2-second single-channel epoch $\mathbf{x}$. A high-resolution spectral analysis is performed by computing the Fast Fourier Transform (FFT) of $\mathbf{x}$ with a zero-padding factor to achieve a frequency resolution of approximately 0.025 Hz. From the complex spectrum $X(f)$, the phase at f_c is estimated by identifying the spectral peak within a narrow band of ± 0.05 Hz around f_c :

$$\phi_c = \angle X(f_{\text{peak}}), \quad \text{where } f_{\text{peak}} = \arg \max_{f \in [f_c - 0.05, f_c + 0.05]} |X(f)| \quad (5)$$

This local peak-picking approach provides robustness against small frequency drifts and spectral leakage.

Directly using the phase angles ϕ_c as features is suboptimal for standard classifiers due to their circular nature: the discontinuity at $\pm\pi$ creates an artificial boundary in the feature space. To circumvent this, each estimated phase is mapped onto the unit circle via its sine and cosine components:

$$\mathbf{p}_c = [\cos(\phi_c), \sin(\phi_c)]^T \quad (6)$$

This transformation provides a unique, linear representation for every phase angle.

For the three candidate frequencies, this process yields six phase-derived features ($\mathbf{p}_{c1}, \mathbf{p}_{c2}, \mathbf{p}_{c3}$). These are concatenated with the original 40-dimensional FBCCA feature vector $\tilde{\rho}$, resulting in a final, enriched feature vector $\mathbf{v}$ of dimension 46:

$$\mathbf{v} = [\tilde{\rho}, \mathbf{p}_{c1}, \mathbf{p}_{c2}, \dots, \mathbf{p}_{cn}]^T \quad (7)$$

This combined vector $\mathbf{v}$ captures both the harmonic strength (via FBCCA) and the precise phase information (via the cascaded estimator) of the SSVEP response, providing a comprehensive representation for target identification.

3.2.3 Classification Strategy

3.2.3.1 Classifier Selection and Pipeline

Target identification was performed using a Support Vector Machine (SVM) classifier with a non-linear Radial Basis Function (RBF) kernel. This choice is motivated by literature demonstrating that SVMs can effectively model the complex, non-linear relationships present in SSVEP feature spaces and often outperform linear classifiers like Linear Discriminant Analysis [1, 2]. The classification was implemented using a pipeline that sequentially applies feature standardization and SVM classification:

1. **Standardization:** Scale each feature dimension to have zero mean and unit variance. This preprocessing step is essential for the RBF kernel, which is sensitive to the scale of the input data.
2. **SVC:** The core classifier with an RBF kernel. The regularization parameter was set to $C = 1.0$ and the kernel coefficient to $\gamma = 0.01$. These values provide a balance between model complexity and generalization.

3.2.3.2 Evaluation Protocol: Leave-One-Subject-Out Cross-Validation

To obtain a robust and subject-independent estimate of system performance, a Leave-One-Subject-Out (LOSO) cross-validation scheme was employed. This method rigorously tests the generalizability of the feature extraction pipeline to novel users. For each of the 35 subjects in the dataset:

- The feature vectors from all trials of that subject were held out as the test set.
- The feature vectors from the remaining 34 subjects were aggregated to form the training set.
- The pipeline (scaler + SVM) was trained on the training set and used to predict the labels for the held-out test subject.

This process was repeated 35 times, ensuring every subject served as the test subject once. The final reported performance metrics are the averages across all 35 test folds.

3.3 Experimental Design

To isolate and quantify the impact of the proposed innovations, channel reduction and phase feature integration, four distinct experimental conditions were evaluated using the LOSO cross-validation protocol.

- A: 9-channel FBCCA (Baseline).** This condition replicates the state-of-the-art multi-channel approach [4], using the FBCCA features from all nine parietal-occipital channels without phase information. It serves as the performance upper bound for a calibrated, multi-electrode system.
- B: 9-channel FBCCA + Phase.** This condition augments the multi-channel baseline with the proposed phase features to evaluate the pure benefit of phase information without the confounding factor of channel reduction.
- C: Single-channel FBCCA.** This condition assesses the performance cost of moving to a practical single-channel setup. The single best channel among the nine candidate parietal-occipital electrodes was selected based on the highest mean accuracy achieved on the training folds during the LOSO cross-validation.
- D: Single-channel FBCCA + Phase (Proposed Method).** This is the complete pipeline, combining the practicality of a single channel with the enriched feature set of FBCCA and explicit phase features.

The comparisons between these conditions allow for a clear interpretation: A vs. B measures the gain from phase features in an optimal multi-channel setting; A vs. C quantifies the performance penalty of using only one channel; C vs. D reveals how much of that penalty can be recovered by adding phase information; and finally, B vs. D illustrates the trade-off between a high-performance multi-channel system and a practical single-channel alternative.

3.4 Performance Metrics and Statistical Analysis

3.4.1 Primary Performance Metrics

The primary metrics for evaluating each condition were classification accuracy and the derived Information Transfer Rate (ITR). ITR (in bits per minute) was calculated using the standard formula [13]:

$$\text{ITR} = \frac{60}{T_{\text{total}}} \left[\log_2 M + P \log_2 P + (1 - P) \log_2 \left(\frac{1 - P}{M - 1} \right) \right], \quad (8)$$

where $M = 40$ is the number of classes, P is the classification accuracy, and T_{total} is the total time per selection in seconds. T_{total} accounts for the data window length ($T_w = 2.0$ s), gaze shifting time ($T_{\text{break}} = 0.5$ s), visual latency ($T_{\text{latency}} = 0.14$ s), resulting in $T_{\text{total}} \approx 2.64$ s per selection. For each of the four experimental conditions, the LOSO cross-validation yields 35 accuracy and ITR values (one per test subject). The mean and standard deviation across subjects are reported as the central performance indicators.

3.4.2 Descriptive and Comparative Statistics

To provide a comprehensive view of performance distribution for the four primary conditions (A–D) using the 2.0-second data window, descriptive statistics, including median, minimum, and maximum values, are reported. To statistically evaluate the pairwise differences between conditions, paired-sample tests were employed. Given that the same 35 subjects provided data for all conditions, paired tests are appropriate. For each comparison, the null hypothesis that the mean difference in accuracy between the two conditions is zero was tested. The t-statistic, p-value, and 95% confidence interval for the mean difference are reported.

3.4.3 Analysis of Data Length Effect

To characterize the performance profile of the proposed methods across different operational speeds, a separate analysis was conducted by varying the data window length (T_w). The epoch start time was kept at 0.14 s post-stimulus, and the window length was varied from 0.25 s to 3.5 s in steps of 0.25 s. This analysis was performed for two critical configurations:

1. **Single-channel (Oz) with Phase:** Representing the proposed practical system.
2. **9-channel with Phase:** Representing the multi-channel performance upper bound.

For each configuration and at each window length, the mean classification accuracy across all 35 subjects (via LOSO) was computed, resulting in a time-series of accuracy values. Paired t-tests were implemented to evaluate the pairwise differences between the two methods for each window length.

4 Results

4.1 Single-channel Performance

In the single-channel FBCCA configuration, performance varied substantially across both recording channels and subjects. Fig. 6 summarizes the classification accuracy and information transfer rate (ITR) averaged across subjects for each of the nine parietal–occipital channels.

Among the evaluated channels, POz and Oz yielded the highest overall performance. POz achieved the highest mean classification accuracy using FBCCA alone, with an average accuracy of approximately 62.5 % and a corresponding mean ITR of 60.5 bits/min. Oz showed similar performance, with a mean accuracy of 62.3 % and a mean ITR of 59.7 bits/min. Other occipital and parietal–occipital channels had moderate to lower performance. For PO6, no improvement could be distinguished. The numbers used for these figures are available in the tables in the Appendix.

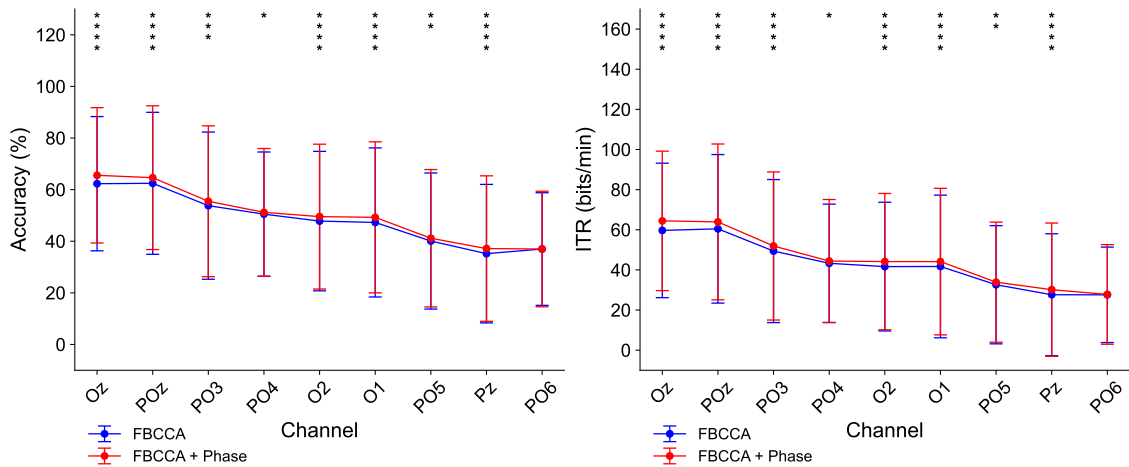


Figure 6: (Left) Classification accuracy and (right) ITR for the FBCCA method and FBCCA + Phase, across different occipital channels. The combined FBCCA and phase method is red, and the FBCCA only method is blue. The error bars indicate standard errors. The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

High inter-subject variability was observed for all channels, as shown by the subject-wise results in Fig. 6. While some subjects achieved near-perfect accuracy using a single channel, others performed close to chance level. Fig. 6 also compares the baseline single-channel FBCCA performance with the proposed phase-validated FBCCA approach.

Across all channels, the inclusion of phase features resulted in a slight, but consistently statistically significant increase in classification accuracy and ITR. For the best-performing channels, the improvement was most pronounced. At Oz, the mean accuracy increased from 62.3 % to 65.5 %. This improvement translated into an increase in mean ITR from 59.7 bits/min to 64.4 bits/min. For POz, phase integration improved mean accuracy from 62.5 % to 64.6 % and increased mean ITR by

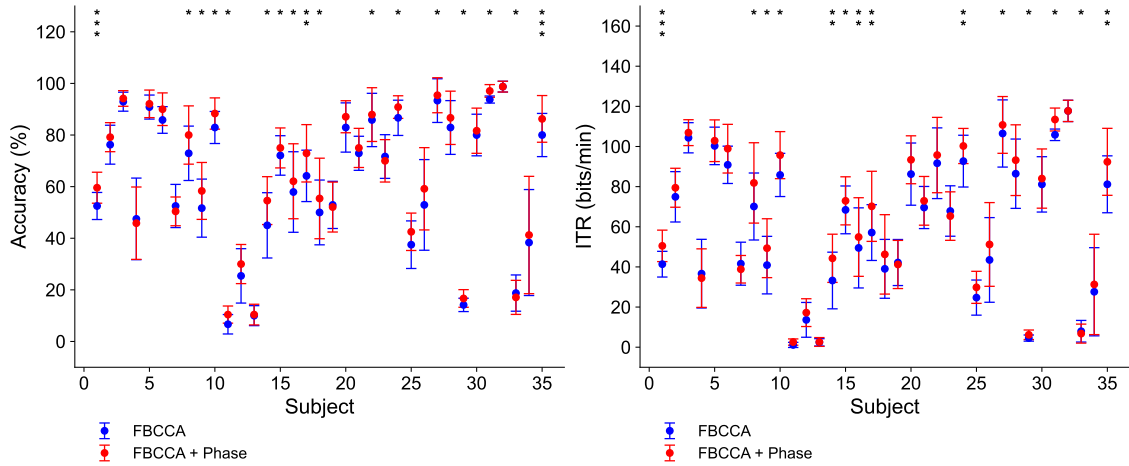


Figure 7: (Left) Classification accuracy and (right) ITR for the FBCCA method and FBCCA + Phase, across different subjects for the best performing channel, Oz. The combined FBCCA and phase method is red, and the FBCCA only method is blue. The error bars indicate standard errors. The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

approximately 3.5 bits/min. Across most channels, integration of phase feature resulted in a mean accuracy increase, ranging from very little for the PO4 channel to approximately 2 - 3 percentage points for the strongest performing channels, Oz, POz, and O1.

Subject-wise results for the Oz channel are shown in Fig. 7 shows accuracy and ITR for the overall best-performing channel, Oz. For the most of subjects, the FBCCA + Phase approach improved performance relative to FBCCA alone, although the improvement increase varied substantially across individuals. Across channels and subjects, the inclusion of phase features lead to a consistent increase in mean accuracy and ITR in the single-channel setting. While the magnitude of the improvement varied across individuals and electrodes, the increase was persistent.

4.1.1 Effect of Time-Window Length

The effect of time-window length T_w on single channel performance of the Oz electrode is shown in Fig 8. For both FBCCA and FBCCA + Phase, classification accuracy increased steadily with increasing T_w , going from near-chance levels to high performance at longer windows. At shorter window lengths, the introduction of the phase components yielded higher performance gains. At $T_w = 1.00$ s, mean accuracy increased from 31.56 ± 18.37 to 39.56 ± 22.18 , an +8.00 percentage points increase in accuracy. At the default window length $T_w = 2.00$ s, the FBCCA + Phase method retained its increase in accuracy, with a +3.08 percentage points and ITR by +4.63 bits/min. As T_w increased beyond 2.5 s, the accuracy and ITR improvement decreased, although the difference between methods remained statistically significant across the window lengths.

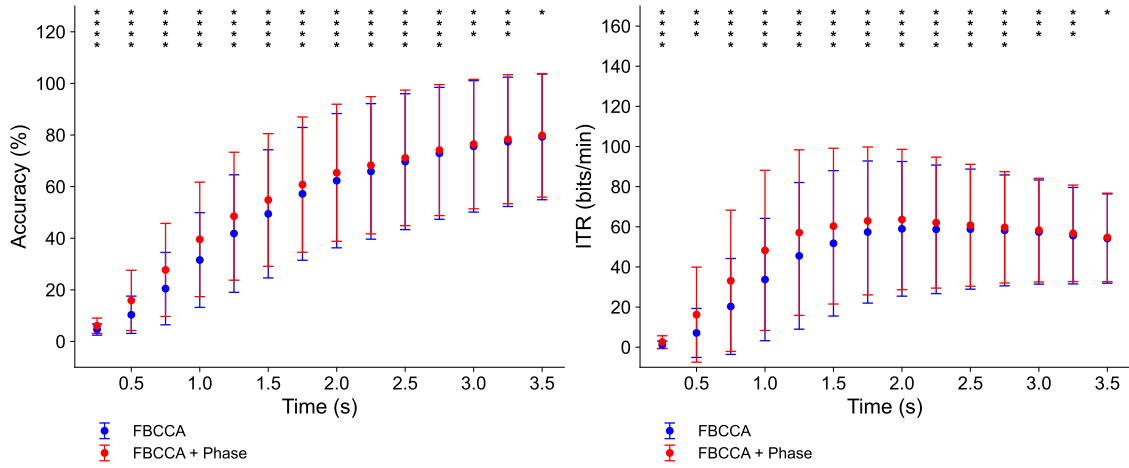


Figure 8: (Left) Classification accuracy and (right) ITR for the FBCCA method and FBCCA + Phase, from a gridsearch for the performance of channel Oz over the length of the time window T_w . The combined FBCCA and phase method is red, and the FBCCA only method is blue. The error bars indicate standard errors. The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

4.2 9-channel Performance

In the multi-channel configuration, performance for the FBCCA method was high across subjects, with several subjects achieving accuracies close to $\sim 100\%$, and only a few subjects showing accuracies $\sim 55\%$. This is illustrated in Fig. 9. Subjects with lower classification accuracy exhibited higher standard deviations, while subjects with higher accuracy showed more stable performance across trials. ITR performs similarly, but with a wider spread.

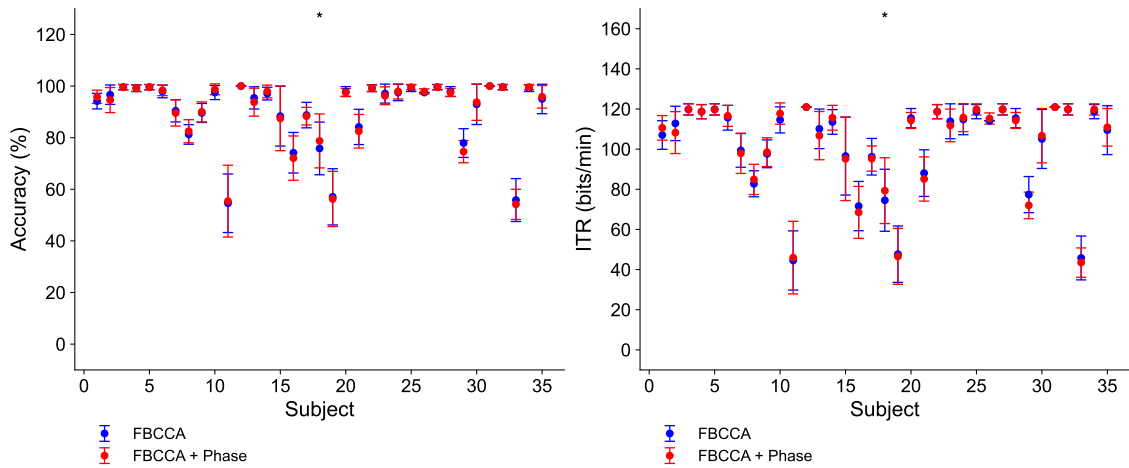


Figure 9: (Left) Classification accuracy and (right) ITR for the FBCCA method and FBCCA + Phase, across different subjects for all 9 recommended occipital channels. The combined FBCCA and phase method is red, and the FBCCA only method is blue. The error bars indicate standard errors. The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

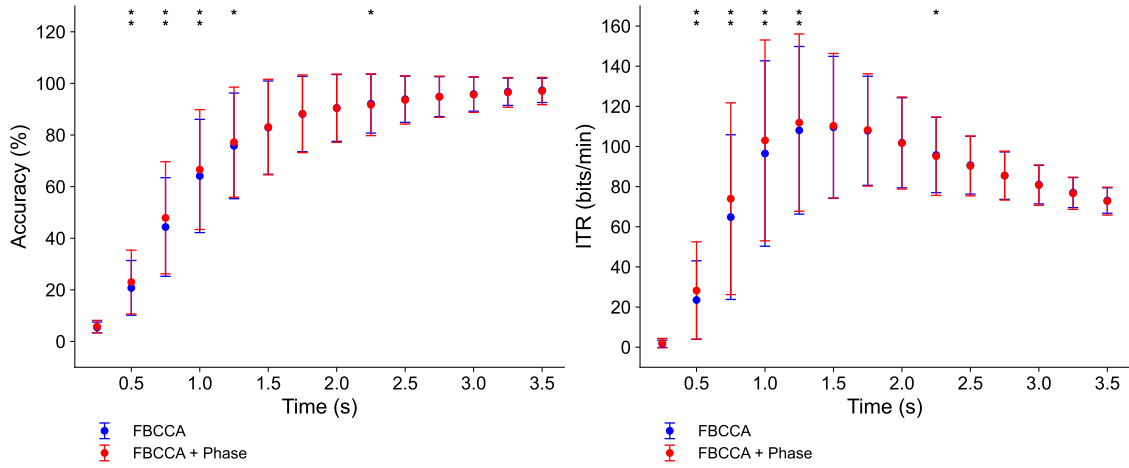


Figure 10: (Left) Classification accuracy and (right) ITR for the FBCCA method and FBCCA + Phase, from a grid search for the performance of all 9 channels over the length of the time window T_w . The combined FBCCA and phase method is red, and the FBCCA only method is blue. The error bars indicate standard errors. The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

Performance for the FBCCA + Phase method was also quite high across subjects, having quite similar results in comparison to the FBCCA alone, at times performing slightly lower and other times performing slightly higher than the reference method. When comparing the overall performance for the two methods across all subjects, trials, and blocks, they perform the same at 90.38% accuracy, and 101.72 ITR. Fig. 9 shows the subject-wise comparison between the two methods. This is also available in Table A9.

4.2.1 Effect of Time-Window Length

The influence of the time-window length in the 9-channel configuration is shown in Fig. 10. Accuracy and ITR increase with window length up to 1.25 s, after which the parameters saturated at $\sim 95\%$ accuracy and ~ 123 ITR. The FBCCA + Phase method provided small but statistically significant improvements at shorter window lengths, where the differences between FBCCA and FBCCA + Phase diminished over longer time windows.

5 Discussion

The primary goal of this study was to assess whether explicit phase information, inherent in JFPM-coded stimuli, could enhance the performance of a single-channel SSVEP-BCI based on the FBCCA framework. The results presented in Fig. 6, Tables A1–A2, and the grid-search analysis (Fig. 8, Tables A7–A8) provide a clear and affirmative answer, with revealing nuances.

First, the adaptation of the multi-channel FBCCA method to a single channel proved successful. The 9-channel baseline (Condition A) achieved an accuracy of 90.38% (see Appendix Table A5), aligning with the high performance reported in the original FBCCA study [4] and validating our implementation. When reduced to a single-channel (Condition C), performance dropped, quantifying the expected cost of sacrificing spatial information. A key observation is that for the FBCCA-only method, the posterior midline site POz (62.45%) slightly outperformed the primary visual cortex site Oz (62.30%) as the best single channel (Table A1).

Critically, the integration of the cascaded phase feature module (Condition D) yielded a statistically significant improvement and altered the optimal channel ranking. For the Oz channel, accuracy increased by 3.8% ($p < 10^{-6}$), reaching 65.38%. This improvement was sufficient to make Oz the top-performing single channel when phase features were included, overtaking POz (Table A1). The shift suggests that the phase signal extracted at Oz is more consistent and discriminative than at POz, allowing the primary visual cortex site to fully leverage the combined frequency-phase information. The phase feature provided a significant boost ($p < 0.0001$) across all other parietal-occipital channels as well (Table A1), with the sole exception of PO4 and PO6, where phase provided little to no benefit. This indicates the robustness of the phase feature, though it can be negated by excessively low channel-specific SNR.

The most revealing finding comes from the grid-search analysis (Fig. 8, Table A7). The absolute benefit of the phase feature (Δ Accuracy) was largest ($\sim 8\%$) at intermediate window lengths (1.0–1.25 s) and remained statistically significant ($p < 0.0001$) across the entire range from 0.25 s to 2.75 s. This demonstrates that phase information is not merely a supplement for long, stable recordings but is particularly valuable for enabling reliable classification at shorter, more practical data lengths aimed at high ITR. The phase feature effectively makes the system more data-efficient.

Conversely, adding phase features to the 9-channel system (Condition B) did not yield a meaningful accuracy increase over the baseline (remaining at $\sim 90\%$, see Tables A9 & A10). This is an expected and important result: when a system already has near-optimal spatial filtering, the marginal discriminative gain from an additional feature domain is minimal. This underscores that the primary value of the phase feature is in resource-constrained scenarios, such as single-channel setups, where every bit of information is critical to mitigate the loss of spatial diversity.

The main limitation of this study is the high inter-subject variability, evident in the large standard deviations in subject-wise results (Fig. 7, Table A3). This "BCI illiteracy" phenomenon is well-documented and reflects the natural variation in SSVEP response strength across individuals. Our use of a globally optimal channel (Oz) for the population is a practical compromise but does not address individual neurophysiological differences, which likely contributes to this variability.

The anomalous result for PO4 and PO6 channels, where phase provided no benefit (Table A1), warrants further investigation. Since PO4 and PO6 are located over the right hemisphere, and all subjects in this study were right-handed, it is possible that the signal at these sites is more susceptible to noise or that volume conduction effects cause phase distortion at this lateral location, making the estimated phase an unreliable feature.

5.1 Related Work

Others have proposed extensions of the CCA methods for SSVEP classification. Yin et al introduced in 2022 a filterbank temporally informed individual template CCA method that include frequency filterbank information, subject specific templates, and temporal structure sequentially [15]. While the method relied on multichannel information, it demonstrated the value of including other complementary information than frequency alone for subject-adaptive purposes.

Alternatively, deep learning approaches such as convolutional neural network could have been applied for its subject and task adaptive capabilities. Safari et al suggested a CNN based classifier operating directly on single channel EEG data from benchmark dataset, showing decent performance outside using feature and classifier design. While neural network can be used to learn frequency and phase related features, they require larger training datasets, more complex computations, and are less transparent compared to traditional signal processing pipelines [10].

5.2 Learning Values

From working on this project, we confirmed that phase information can add discriminative value to jointly phase and frequency modulated SSVEP stimuli, even with a low-complexity single-channel setting. By extending a well-defined standard FBCCA pipeline with a light phase validation step, we could observe robust improvements in classification accuracy and information transfer rate across channels, subjects, and time window lengths. This shows that the phase information can be used to support low-complexity single-channel settings without unnecessary complications.

One of the learning outcomes included the strong dependence of single channel performance on electrode placement and time window length. Occipital and parietal occipital channels such as Oz and POz produced by far the highest performance, while larger inter-subject variability remained

present across all channels. Another learning outcome was that the phase features provided the largest benefit at the shorter time windows, where frequency based discriminative information alone was weaker. These takeaways can be important for future designs for low-complexity systems, which are relevant for real time and more affordable BCI implementations.

While the FBCCA + Phase method consistently improves single channel performance, the sheer increase of the improvement was modest compared to overall variability between subject. In the 9-channel configuration, the addition of the phase features did not lead to a meaningful overall performance increase. This suggests that when sufficient spatial information is available, FBCCA already captures most of the discriminating information, leaving less room for further additions from phase validation.

5.3 Future Work

There are many possible extensions of this work. First, subject-adaptive calibration on both phase and frequency could be explored. Here, phase references tuned to the individual users, could be useful in subjects that have low classification performance and high inter-subject variability. This approach could improve the method's robustness and enhance the single-channel performance at the cost of spending time pre-trial on recording data for calibration. Other variations of the canonical correlation analysis have been proposed to improve subject-specific SSVEP decoding performance, especially for short time windows and for greater inter-subject variability.

Secondly, the current implementation applies phase validation in sequence after the FBCCA by selecting the top candidate frequencies and refining the decision using these phase features. Another approach would be to extract phase and frequency features in parallel, ensuring an equal amount of features from both domains in a single classifier. A combination like that would treat phase and frequency information symmetrically and could allow the classifier to learn joint decision boundaries, potentially improving performance in short time-window and low-SNR conditions, as with single-channel configurations.

6 Conclusion

This study successfully demonstrated that a single-channel SSVEP-BCI can achieve robust performance by combining harmonic frequency analysis (via an adapted FBCCA) and explicit phase information from JFPM-coded stimuli. The proposed cascaded phase feature extraction module provided a statistically significant accuracy boost across nearly all recording sites and was especially valuable at shorter data lengths, enhancing information transfer rates. Notably, the inclusion of phase features altered the optimal channel ranking, enabling the primary visual cortex site (Oz) to outperform other sites, highlighting the neurophysiological relevance of the extracted phase signal.

We learned that while spatial filtering (multiple channels) is the most powerful tool for SSVEP decoding, intelligent feature engineering can recover a substantial portion of the lost performance when hardware simplicity is paramount. The phase component, often underutilized, proved to be a stable and discriminative feature that effectively compensates for the lack of spatial diversity in a single-channel setup.

In conclusion, this work provides a methodological advancement that makes high-performance SSVEP-based communication more accessible. By refining signal processing rather than adding hardware complexity, we take a meaningful step towards practical, user-friendly, and wearable BCI systems for assistive technology.

References

- [1] Giovanni Acampora, Pasquale Trinchese, and Autilia Vitiello. “Classifying EEG Signals in Single-Channel SSVEP-based BCIs through Support Vector Machine”. In: *Conference Proceedings - IEEE International Conference on Systems, Man and Cybernetics 2020-October* (Oct. 2020), pp. 2305–2310. ISSN: 1062922X. DOI: 10.1109/SMC42975.2020.9282940.
- [2] Phairot Autthasan et al. “A Single-Channel Consumer-Grade EEG Device for Brain-Computer Interface: Enhancing Detection of SSVEP and Its Amplitude Modulation”. In: *IEEE Sensors Journal* 20.6 (Mar. 2020), pp. 3366–3378. ISSN: 15581748. DOI: 10.1109/JSEN.2019.2958210.
- [3] Mario Corral Bolanos, Sheyla Barrado Ballesteros, and Sadasivan Puthusserypady. “Filter bank approach for enhancement of supervised Canonical Correlation Analysis methods for SSVEP-based BCI spellers”. In: *Proceedings of the Annual International Conference of the IEEE Engineering in Medicine and Biology Society, EMBS 2021-January* (2021), pp. 337–340. ISSN: 1557170X. DOI: 10.1109/EMBC46164.2021.9630838.
- [4] Xiaogang Chen et al. “Filter bank canonical correlation analysis for implementing a high-speed SSVEP-based brain-computer interface”. In: *Journal of Neural Engineering* 12.4 (Aug. 2015). ISSN: 17412552. DOI: 10.1088/1741-2560/12/4/046008.
- [5] Sanduni P. Karunasena et al. “Single-Channel EEG SSVEP-based BCI for Robot Arm Control”. In: *2021 IEEE Sensors Applications Symposium (SAS)*. IEEE, Aug. 2021, pp. 1–6. ISBN: 978-1-7281-9431-8. DOI: 10.1109/SAS51076.2021.9530189. URL: <https://ieeexplore.ieee.org/document/9530189/>.
- [6] Chang S. Nam, Anton Nijholt, and Fabien Lotte. *Brain-Computer Interfaces Handbook*. Ed. by Chang S. Nam, Anton Nijholt, and Fabien Lotte. Boca Raton : Taylor & Francis, CRC Press, 2018.: CRC Press, Jan. 2018. ISBN: 9781351231954. DOI: 10.1201/9781351231954.
- [7] Sadasivan Puthusserypady. *Lecture 3: Signal Processing in BCIs*. 2026.
- [8] Lu Qiu and Wenya Nan. “Brain Network Constancy and Participant Recognition: an Integrated Approach to Big Data and Complex Network Analysis”. In: *Frontiers in Psychology* 11 (June 2020). ISSN: 16641078. DOI: 10.3389/FPSYG.2020.01003.
- [9] Preetha S. and Sasikala M. “Design of asynchronous low-complexity SSVEP-based brain control interface speller”. In: *Computers in Biology and Medicine* 190 (May 2025), p. 110062. ISSN: 0010-4825. DOI: 10.1016/J.COMPBIOMED.2025.110062.

- [10] Farzad Saffari and Ali Khadem. “Single-channel selection for detecting steady-state visual evoked potentials in a brain-computer interface speller”. In: *Frontiers in Biomedical Technologies* 8.3 (2021), pp. 155–160. ISSN: 23455837. DOI: 10.18502/fbt.v8i3.7109.
- [11] Abdullah Talha Sozer. “Enhanced Single Channel SSVEP Detection Method on Benchmark Dataset”. In: *2018 15th International Conference on Electrical Engineering, Computing Science and Automatic Control, CCE 2018* (Nov. 2018). DOI: 10.1109/ICEEE.2018.8533933.
- [12] Yijun Wang et al. “A Benchmark Dataset for SSVEP-Based Brain-Computer Interfaces”. In: *IEEE transactions on neural systems and rehabilitation engineering : a publication of the IEEE Engineering in Medicine and Biology Society* 25.10 (Oct. 2017), pp. 1746–1752. ISSN: 1558-0210. DOI: 10.1109/TNSRE.2016.2627556. URL: <https://pubmed.ncbi.nlm.nih.gov/27849543/>.
- [13] Jonathan Wolpaw and Elizabeth Winter Wolpaw. *Brain-Computer Interfaces: Principles and Practice*. Oxford University Press, Jan. 2012. ISBN: 9780195388855. DOI: 10.1093/acprof:oso/9780195388855.001.0001.
- [14] Le Wu et al. “Signal Processing for Brain-Computer Interfaces: A review and current perspectives”. In: *IEEE Signal Processing Magazine* 40.5 (July 2023), pp. 80–91. ISSN: 15580792. DOI: 10.1109/MSP.2023.3278074.
- [15] Xiangguo Yin and Mingxing Lin. “Multi-information improves the performance of CCA-based SSVEP classification”. In: *Cognitive Neurodynamics* 18.1 (Feb. 2023), p. 165. ISSN: 18714099. DOI: 10.1007/S11571-022-09923-X. URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10881948/>.

Appendices

A Statistical Test Results

Single-Channel Configuration: Ranking

Table A1: Classification accuracy (%) for the ranked single channels across all subjects. The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

Channel	FBCCA ($\mu \pm \sigma$ %)	FBCCA + Phase ($\mu \pm \sigma$ %)	Δ (%)	p value	Significance
Oz	62.30 $\pm$ 26.01	65.55 $\pm$ 26.22	+3.25	2.06×10^{-7}	****
POz	62.45 $\pm$ 27.51	64.63 $\pm$ 27.85	+2.18	1.80×10^{-8}	****
PO3	53.79 $\pm$ 28.52	55.49 $\pm$ 29.23	+1.70	2.37×10^{-4}	***
PO4	50.50 $\pm$ 24.08	51.20 $\pm$ 24.74	+0.70	2.80×10^{-2}	*
O2	47.81 $\pm$ 27.02	49.55 $\pm$ 28.06	+1.74	8.51×10^{-5}	****
O1	47.29 $\pm$ 28.89	49.26 $\pm$ 29.27	+1.97	1.01×10^{-5}	****
PO5	40.08 $\pm$ 26.37	41.17 $\pm$ 26.63	+1.09	2.19×10^{-3}	**
Pz	35.19 $\pm$ 26.83	37.15 $\pm$ 28.19	+1.96	1.69×10^{-6}	****
PO6	36.98 $\pm$ 21.82	36.99 $\pm$ 22.39	+0.01	9.73×10^{-1}	

Table A2: Information transfer rate (bits/min) for the ranked single channels across all subjects. The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

Channel	FBCCA ($\mu \pm \sigma$ bits/min)	FBCCA + Phase ($\mu \pm \sigma$ bits/min)	Δ (bits/min)	p value	Significance
Oz	59.67 $\pm$ 33.51	64.43 $\pm$ 34.74	+4.76	1.45×10^{-7}	****
POz	60.47 $\pm$ 37.00	63.93 $\pm$ 38.81	+3.46	6.17×10^{-7}	****
PO3	49.39 $\pm$ 35.63	51.92 $\pm$ 36.90	+2.53	4.95×10^{-5}	****
PO4	43.30 $\pm$ 29.48	44.42 $\pm$ 30.66	+1.12	1.06×10^{-2}	*
O2	41.65 $\pm$ 32.06	44.15 $\pm$ 33.95	+2.50	4.76×10^{-5}	****
O1	41.71 $\pm$ 35.54	44.15 $\pm$ 36.51	+2.44	1.88×10^{-5}	****
PO5	32.62 $\pm$ 29.44	33.90 $\pm$ 29.92	+1.28	2.55×10^{-3}	**
Pz	27.64 $\pm$ 30.37	30.15 $\pm$ 33.21	+2.51	8.40×10^{-5}	****
PO6	27.59 $\pm$ 23.78	27.79 $\pm$ 24.81	+0.20	6.22×10^{-1}	

Single-Channel Configuration: Oz

Table A3: Accuracy (%) for the Oz channel reported per subject. The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

Subject	FBCCA ($\mu \pm \sigma$ %)	FBCCA + Phase ($\mu \pm \sigma$ %)	Δ (%)	p value	Significance
1	52.50 $\pm$ 5.24	59.58 $\pm$ 6.00	+7.08	8.82×10^{-4}	***
2	76.25 $\pm$ 7.54	79.17 $\pm$ 5.63	+2.92	1.10×10^{-1}	
3	92.92 $\pm$ 3.68	94.17 $\pm$ 3.03	+1.25	7.56×10^{-2}	
4	47.50 $\pm$ 15.81	45.83 $\pm$ 14.02	−1.67	2.86×10^{-1}	
5	90.83 $\pm$ 4.65	92.08 $\pm$ 5.34	+1.25	4.15×10^{-1}	
6	85.83 $\pm$ 5.16	90.00 $\pm$ 6.32	+4.17	1.17×10^{-1}	
7	52.50 $\pm$ 8.37	50.42 $\pm$ 5.57	−2.08	2.89×10^{-1}	
8	72.92 $\pm$ 10.54	80.00 $\pm$ 11.29	+7.08	3.02×10^{-2}	*
9	51.67 $\pm$ 11.25	58.33 $\pm$ 11.03	+6.67	1.71×10^{-2}	*
10	82.92 $\pm$ 6.21	88.33 $\pm$ 6.06	+5.42	1.55×10^{-2}	*
11	6.67 $\pm$ 3.76	10.42 $\pm$ 3.32	+3.75	3.01×10^{-2}	*
12	25.42 $\pm$ 10.54	30.00 $\pm$ 7.58	+4.58	1.10×10^{-1}	
13	10.00 $\pm$ 3.87	10.42 $\pm$ 4.01	+0.42	7.41×10^{-1}	
14	45.00 $\pm$ 12.65	54.58 $\pm$ 9.28	+9.58	1.29×10^{-2}	*
15	72.08 $\pm$ 7.65	75.00 $\pm$ 7.75	+2.92	3.35×10^{-2}	*
16	57.92 $\pm$ 15.61	62.08 $\pm$ 14.53	+4.17	1.08×10^{-2}	*
17	64.17 $\pm$ 9.96	72.92 $\pm$ 11.11	+8.75	4.60×10^{-3}	**
18	50.00 $\pm$ 12.55	55.42 $\pm$ 15.61	+5.42	4.82×10^{-2}	*
19	52.92 $\pm$ 9.14	52.08 $\pm$ 9.67	−0.83	3.63×10^{-1}	
20	82.92 $\pm$ 9.54	87.08 $\pm$ 6.21	+4.17	1.53×10^{-1}	
21	72.92 $\pm$ 6.60	75.00 $\pm$ 7.58	+2.08	9.25×10^{-2}	
22	85.83 $\pm$ 10.33	87.92 $\pm$ 10.42	+2.08	4.22×10^{-2}	*
23	71.67 $\pm$ 8.47	70.00 $\pm$ 8.22	−1.67	1.02×10^{-1}	
24	86.67 $\pm$ 6.83	90.83 $\pm$ 4.38	+4.17	1.08×10^{-2}	*
25	37.50 $\pm$ 9.22	42.50 $\pm$ 7.25	+5.00	6.68×10^{-2}	
26	52.92 $\pm$ 17.57	59.17 $\pm$ 15.94	+6.25	1.81×10^{-1}	
27	93.33 $\pm$ 8.47	95.42 $\pm$ 6.79	+2.08	4.22×10^{-2}	*
28	82.92 $\pm$ 10.42	86.67 $\pm$ 10.33	+3.75	5.99×10^{-2}	
29	14.17 $\pm$ 2.58	16.67 $\pm$ 3.42	+2.50	1.17×10^{-2}	*
30	80.00 $\pm$ 8.06	81.67 $\pm$ 8.76	+1.67	5.01×10^{-1}	
31	93.75 $\pm$ 1.37	97.08 $\pm$ 2.46	+3.33	2.50×10^{-2}	*
32	98.75 $\pm$ 2.09	98.75 $\pm$ 2.09	0.00	–	
33	18.75 $\pm$ 7.03	17.08 $\pm$ 6.60	−1.67	2.50×10^{-2}	*
34	38.33 $\pm$ 20.53	41.25 $\pm$ 22.73	+2.92	1.80×10^{-1}	
35	80.00 $\pm$ 8.37	86.25 $\pm$ 9.05	+6.25	7.46×10^{-4}	****

Table A4: Information transfer rate (bits/min) for the Oz channel per subject. The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

Subject	FBCCA ($\mu \pm \sigma$ bits/min)	FBCCA + Phase ($\mu \pm \sigma$ bits/min)	Δ (bits/min)	p value	Significance
1	41.36 $\pm$ 6.40	50.49 $\pm$ 7.84	+9.13	8.44×10^{-4}	****
2	74.93 $\pm$ 12.54	79.44 $\pm$ 9.70	+4.51	1.09×10^{-1}	
3	104.34 $\pm$ 7.52	106.90 $\pm$ 6.45	+2.56	7.70×10^{-2}	
4	36.64 $\pm$ 17.11	34.42 $\pm$ 14.58	-2.22	2.72×10^{-1}	
5	100.30 $\pm$ 9.34	102.87 $\pm$ 10.38	+2.57	3.90×10^{-1}	
6	90.87 $\pm$ 9.30	98.91 $\pm$ 12.13	+8.04	1.20×10^{-1}	
7	41.60 $\pm$ 10.72	38.84 $\pm$ 6.86	-2.76	2.74×10^{-1}	
8	70.10 $\pm$ 16.69	81.85 $\pm$ 20.02	+11.75	3.32×10^{-2}	*
9	40.89 $\pm$ 14.33	49.33 $\pm$ 14.67	+8.44	1.64×10^{-2}	*
10	85.82 $\pm$ 10.76	95.69 $\pm$ 11.72	+9.87	1.47×10^{-2}	*
11	1.13 $\pm$ 1.28	2.56 $\pm$ 1.56	+1.43	5.75×10^{-2}	
12	13.62 $\pm$ 8.67	17.22 $\pm$ 6.92	+3.60	1.04×10^{-1}	
13	2.42 $\pm$ 1.83	2.61 $\pm$ 2.19	+0.19	7.58×10^{-1}	
14	33.23 $\pm$ 14.11	44.29 $\pm$ 12.01	+11.06	6.88×10^{-3}	**
15	68.42 $\pm$ 11.91	72.92 $\pm$ 11.98	+4.50	3.44×10^{-2}	*
16	49.47 $\pm$ 19.96	54.89 $\pm$ 19.58	+5.42	9.55×10^{-3}	**
17	57.11 $\pm$ 13.90	70.18 $\pm$ 17.47	+13.07	6.98×10^{-3}	**
18	39.04 $\pm$ 14.67	46.25 $\pm$ 19.78	+7.21	6.73×10^{-2}	
19	42.19 $\pm$ 11.47	41.22 $\pm$ 11.99	-0.97	3.68×10^{-1}	
20	86.24 $\pm$ 15.49	93.38 $\pm$ 11.94	+7.14	1.35×10^{-1}	
21	69.59 $\pm$ 10.49	72.93 $\pm$ 12.14	+3.34	7.73×10^{-2}	
22	91.66 $\pm$ 17.62	95.73 $\pm$ 18.77	+4.07	5.76×10^{-2}	
23	67.85 $\pm$ 12.56	65.33 $\pm$ 12.08	-2.52	9.51×10^{-2}	
24	92.70 $\pm$ 12.90	100.25 $\pm$ 8.75	+7.55	8.80×10^{-3}	**
25	24.71 $\pm$ 8.75	29.82 $\pm$ 7.98	+5.11	5.76×10^{-2}	
26	43.48 $\pm$ 21.09	51.19 $\pm$ 20.86	+7.71	1.49×10^{-1}	
27	106.48 $\pm$ 16.74	110.74 $\pm$ 14.14	+4.26	3.05×10^{-2}	*
28	86.44 $\pm$ 17.27	93.18 $\pm$ 17.62	+6.74	6.78×10^{-2}	
29	4.55 $\pm$ 1.53	6.19 $\pm$ 2.37	+1.64	2.02×10^{-2}	*
30	81.13 $\pm$ 13.79	84.02 $\pm$ 14.76	+2.89	4.73×10^{-1}	
31	105.82 $\pm$ 2.86	113.44 $\pm$ 5.71	+7.62	2.73×10^{-2}	*
32	117.73 $\pm$ 5.31	117.73 $\pm$ 5.31	0.00	–	
33	7.95 $\pm$ 5.39	6.77 $\pm$ 4.74	-1.18	3.16×10^{-2}	*
34	27.62 $\pm$ 21.96	31.27 $\pm$ 25.00	+3.65	1.55×10^{-1}	
35	81.16 $\pm$ 14.18	92.36 $\pm$ 16.69	+11.20	1.63×10^{-3}	**

9-channel Configuration

Table A5: Multichannel (9-channel) classification accuracy (%) per subject. The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

Subject	FBCCA ($\mu \pm \sigma$ %)	FBCCA + Phase ($\mu \pm \sigma$ %)	Δ (%)	p value	Significance
1	94.17 $\pm$ 3.03	95.83 $\pm$ 2.58	+1.67	0.102	
2	96.67 $\pm$ 3.76	94.58 $\pm$ 4.85	−2.08	0.093	
3	99.58 $\pm$ 1.02	99.58 $\pm$ 1.02	0.00	—	
4	99.17 $\pm$ 1.29	99.17 $\pm$ 1.29	0.00	—	
5	99.58 $\pm$ 1.02	99.58 $\pm$ 1.02	0.00	—	
6	97.92 $\pm$ 2.46	98.33 $\pm$ 2.04	+0.42	0.695	
7	90.42 $\pm$ 4.31	89.58 $\pm$ 5.10	−0.83	0.530	
8	81.25 $\pm$ 3.79	82.50 $\pm$ 4.47	+1.25	0.542	
9	89.58 $\pm$ 3.68	90.00 $\pm$ 3.87	+0.42	0.695	
10	97.50 $\pm$ 2.74	98.75 $\pm$ 2.09	+1.25	0.076	
11	54.58 $\pm$ 11.34	55.42 $\pm$ 13.91	+0.83	0.771	
12	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	0.00	—	
13	95.42 $\pm$ 4.31	93.75 $\pm$ 5.42	−1.67	0.102	
14	97.08 $\pm$ 2.46	97.92 $\pm$ 2.46	+0.83	0.175	
15	88.33 $\pm$ 11.58	87.50 $\pm$ 12.55	−0.83	0.363	
16	74.17 $\pm$ 7.85	72.08 $\pm$ 8.58	−2.08	0.141	
17	88.75 $\pm$ 4.94	88.33 $\pm$ 3.42	−0.42	0.611	
18	75.83 $\pm$ 10.21	78.75 $\pm$ 10.46	+2.92	0.013	*
19	57.08 $\pm$ 10.89	56.25 $\pm$ 10.69	−0.83	0.816	
20	97.92 $\pm$ 1.88	97.50 $\pm$ 1.58	−0.42	0.363	
21	84.17 $\pm$ 6.83	82.50 $\pm$ 6.52	−1.67	0.363	
22	99.17 $\pm$ 1.29	99.17 $\pm$ 1.29	0.00	—	
23	97.08 $\pm$ 3.68	96.25 $\pm$ 3.45	−0.83	0.175	
24	97.50 $\pm$ 3.16	97.92 $\pm$ 2.92	+0.42	0.363	
25	99.17 $\pm$ 1.29	99.58 $\pm$ 1.02	+0.42	0.363	
26	97.50 $\pm$ 0.00	97.92 $\pm$ 1.02	+0.42	0.363	
27	99.58 $\pm$ 1.02	99.58 $\pm$ 1.02	0.00	—	
28	97.92 $\pm$ 1.88	97.50 $\pm$ 1.58	−0.42	0.363	
29	77.92 $\pm$ 5.57	74.58 $\pm$ 4.31	−3.33	0.102	
30	92.92 $\pm$ 7.81	93.75 $\pm$ 7.03	+0.83	0.175	
31	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	0.00	—	
32	99.58 $\pm$ 1.02	99.58 $\pm$ 1.02	0.00	—	
33	55.83 $\pm$ 8.32	54.17 $\pm$ 5.85	−1.67	0.394	
34	99.17 $\pm$ 1.29	99.58 $\pm$ 1.02	+0.42	0.363	
35	95.00 $\pm$ 5.70	95.83 $\pm$ 4.38	+0.83	0.465	

Table A6: Multichannel (9-channel) ITR (bits/min) reported per subject. The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

Subject	FBCCA ($\mu \pm \sigma$ bits/min)	FBCCA + Phase ($\mu \pm \sigma$ bits/min)	Δ (bits/min)	p value	Significance
1	107.04 $\pm$ 7.13	110.60 $\pm$ 6.14	+3.56	0.105	
2	112.81 $\pm$ 8.56	108.25 $\pm$ 10.44	−4.56	0.103	
3	119.81 $\pm$ 2.79	119.81 $\pm$ 2.79	0.00	—	
4	118.67 $\pm$ 3.53	118.67 $\pm$ 3.53	0.00	—	
5	119.81 $\pm$ 2.79	119.81 $\pm$ 2.79	0.00	—	
6	115.64 $\pm$ 6.18	116.59 $\pm$ 5.21	+0.95	0.728	
7	99.39 $\pm$ 8.39	97.92 $\pm$ 10.02	−1.47	0.563	
8	82.74 $\pm$ 6.48	84.91 $\pm$ 7.53	+2.17	0.524	
9	97.68 $\pm$ 6.93	98.49 $\pm$ 7.16	+0.81	0.675	
10	114.58 $\pm$ 6.50	117.73 $\pm$ 5.31	+3.15	0.079	
11	44.53 $\pm$ 14.74	45.96 $\pm$ 18.09	+1.43	0.697	
12	120.95 $\pm$ 0.00	120.95 $\pm$ 0.00	0.00	—	
13	110.12 $\pm$ 9.86	106.75 $\pm$ 12.03	−3.37	0.095	
14	113.56 $\pm$ 6.14	115.64 $\pm$ 6.18	+2.09	0.177	
15	96.54 $\pm$ 19.42	95.18 $\pm$ 20.79	−1.36	0.436	
16	71.64 $\pm$ 12.28	68.51 $\pm$ 12.94	−3.14	0.137	
17	96.23 $\pm$ 9.12	95.28 $\pm$ 6.29	−0.95	0.535	
18	74.54 $\pm$ 15.45	79.32 $\pm$ 16.39	+4.79	0.019	*
19	47.67 $\pm$ 14.05	46.58 $\pm$ 13.94	−1.09	0.800	
20	115.45 $\pm$ 4.80	114.31 $\pm$ 3.97	−1.14	0.363	
21	88.06 $\pm$ 11.62	85.12 $\pm$ 11.02	−2.94	0.319	
22	118.67 $\pm$ 3.53	118.67 $\pm$ 3.53	0.00	—	
23	113.90 $\pm$ 8.69	111.81 $\pm$ 8.15	−2.09	0.177	
24	114.77 $\pm$ 7.60	115.72 $\pm$ 6.98	+0.95	0.363	
25	118.67 $\pm$ 3.53	119.81 $\pm$ 2.79	+1.14	0.363	
26	114.12 $\pm$ 0.00	115.26 $\pm$ 2.79	+1.14	0.363	
27	119.81 $\pm$ 2.79	119.81 $\pm$ 2.79	0.00	—	
28	115.45 $\pm$ 4.80	114.31 $\pm$ 3.97	−1.14	0.363	
29	77.36 $\pm$ 9.00	71.97 $\pm$ 6.63	−5.40	0.102	
30	105.07 $\pm$ 14.70	106.70 $\pm$ 13.55	+1.63	0.182	
31	120.95 $\pm$ 0.00	120.95 $\pm$ 0.00	0.00	—	
32	119.81 $\pm$ 2.79	119.81 $\pm$ 2.79	0.00	—	
33	45.79 $\pm$ 10.93	43.47 $\pm$ 7.31	−2.31	0.371	
34	118.67 $\pm$ 3.53	119.81 $\pm$ 2.79	+1.14	0.363	
35	109.46 $\pm$ 12.18	110.89 $\pm$ 9.36	+1.43	0.578	

Grid Search: Single-Channel Configuration Oz**Table A7:** Classification accuracy (%) for the Oz channel across different time window lengths t_w . The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

t_w (s)	FBCCA ($\mu \pm \sigma$ %)	FBCCA + Phase ($\mu \pm \sigma$ %)	Δ (%)	p value	Significance
0.25	4.62 ± 2.18	6.10 ± 2.98	+1.48	7.37×10^{-5}	****
0.50	10.35 ± 7.23	15.90 ± 11.68	+5.56	6.19×10^{-7}	****
0.75	20.48 ± 14.02	27.71 ± 18.04	+7.24	1.85×10^{-10}	****
1.00	31.56 ± 18.37	39.56 ± 22.18	+8.00	1.51×10^{-9}	****
1.25	41.81 ± 22.77	48.54 ± 24.79	+6.73	3.16×10^{-9}	****
1.50	49.45 ± 24.83	54.82 ± 25.71	+5.37	1.23×10^{-11}	****
1.75	57.19 ± 25.74	60.77 ± 26.21	+3.58	4.95×10^{-9}	****
2.00	62.30 ± 26.01	65.38 ± 26.56	+3.08	3.22×10^{-7}	****
2.25	65.89 ± 26.26	68.27 ± 26.58	+2.38	1.53×10^{-7}	****
2.50	69.67 ± 26.33	71.13 ± 26.26	+1.46	5.43×10^{-5}	****
2.75	72.90 ± 25.56	74.14 ± 25.38	+1.24	5.01×10^{-7}	****
3.00	75.60 ± 25.46	76.48 ± 25.09	+0.88	4.16×10^{-4}	***
3.25	77.36 ± 25.06	78.37 ± 25.00	+1.01	9.93×10^{-4}	***
3.50	79.27 ± 24.36	79.87 ± 23.95	+0.60	1.41×10^{-2}	*

Table A8: Classification ITR (bits/min) for the Oz channel across different time window lengths t_w . The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

t_w (s)	FBCCA ($\mu \pm \sigma$ bits/min)	FBCCA + Phase ($\mu \pm \sigma$ bits/min)	Δ (bits/min)	p value	Significance
0.25	1.17 ± 1.88	2.56 ± 3.13	+1.39	9.18×10^{-5}	****
0.50	7.09 ± 12.20	16.22 ± 23.69	+9.12	1.14×10^{-4}	***
0.75	20.29 ± 23.90	33.08 ± 35.19	+12.80	5.85×10^{-7}	****
1.00	33.70 ± 30.47	48.22 ± 39.90	+14.52	4.94×10^{-8}	****
1.25	45.51 ± 36.53	57.09 ± 41.25	+11.58	2.67×10^{-8}	****
1.50	51.74 ± 36.23	60.31 ± 38.80	+8.57	3.60×10^{-10}	****
1.75	57.38 ± 35.41	62.90 ± 36.84	+5.52	6.75×10^{-9}	****
2.00	58.98 ± 33.55	63.61 ± 34.97	+4.63	1.65×10^{-7}	****
2.25	58.71 ± 32.01	62.07 ± 32.62	+3.36	7.64×10^{-8}	****
2.50	58.82 ± 29.92	60.74 ± 30.38	+1.92	9.00×10^{-5}	****
2.75	58.20 ± 27.63	59.71 ± 27.74	+1.52	1.05×10^{-6}	****
3.00	57.36 ± 25.94	58.32 ± 25.83	+0.96	5.54×10^{-4}	***
3.25	55.56 ± 24.03	56.73 ± 24.04	+1.16	3.55×10^{-4}	***
3.50	54.12 ± 22.24	54.68 ± 22.06	+0.56	1.45×10^{-2}	*

Grid Search: 9-channel Configuration

Table A9: Classification accuracy (%) for the 9-channel setup across different time window lengths t_w . The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

t_w (s)	FBCCA ($\mu \pm \sigma$ %)	FBCCA + Phase ($\mu \pm \sigma$ %)	Δ (%)	p value	Significance
0.25	5.42 $\pm$ 2.10	5.80 $\pm$ 2.38	+0.38	1.44×10^{-1}	
0.50	20.77 $\pm$ 10.60	23.02 $\pm$ 12.36	+2.25	1.34×10^{-3}	**
0.75	44.36 $\pm$ 19.11	47.92 $\pm$ 21.72	+3.56	2.67×10^{-3}	**
1.00	64.13 $\pm$ 21.94	66.62 $\pm$ 23.21	+2.49	4.97×10^{-3}	**
1.25	75.80 $\pm$ 20.48	77.23 $\pm$ 21.34	+1.43	1.11×10^{-2}	*
1.50	82.86 $\pm$ 18.07	83.10 $\pm$ 18.52	+0.24	5.25×10^{-1}	
1.75	88.13 $\pm$ 14.60	88.23 $\pm$ 15.05	+0.10	6.89×10^{-1}	
2.00	90.50 $\pm$ 12.94	90.38 $\pm$ 13.22	−0.12	5.62×10^{-1}	
2.25	92.13 $\pm$ 11.38	91.73 $\pm$ 11.97	−0.40	2.66×10^{-2}	*
2.50	93.86 $\pm$ 8.96	93.60 $\pm$ 9.35	−0.26	2.13×10^{-1}	
2.75	94.87 $\pm$ 7.73	94.82 $\pm$ 7.98	−0.05	7.81×10^{-1}	
3.00	95.86 $\pm$ 6.62	95.65 $\pm$ 6.87	−0.20	2.41×10^{-1}	
3.25	96.77 $\pm$ 5.29	96.51 $\pm$ 5.75	−0.26	1.35×10^{-1}	
3.50	97.30 $\pm$ 4.70	97.05 $\pm$ 5.29	−0.25	1.47×10^{-1}	

Table A10: Classification ITR (bits/min) for the 9-channel setup across different time window lengths t_w . The Δ column is (FBCCA + Phase) – (FBCCA). The asterisks show significant difference between the methods (*: $p < 0.05$, **: $p < 0.01$, ***: $p < 0.001$, ****: $p < 0.0001$).

t_w (s)	FBCCA ($\mu \pm \sigma$ bits/min)	FBCCA + Phase ($\mu \pm \sigma$ bits/min)	Δ (bits/min)	p value	Significance
0.25	1.67 $\pm$ 1.83	2.06 $\pm$ 2.31	+0.39	1.15×10^{-1}	
0.50	23.50 $\pm$ 19.53	28.24 $\pm$ 24.26	+4.74	1.60×10^{-3}	**
0.75	64.81 $\pm$ 41.03	73.98 $\pm$ 47.77	+9.16	1.99×10^{-3}	**
1.00	96.50 $\pm$ 46.19	103.02 $\pm$ 50.05	+6.52	3.44×10^{-3}	**
1.25	108.05 $\pm$ 41.77	111.89 $\pm$ 44.18	+3.84	2.66×10^{-3}	**
1.50	109.58 $\pm$ 35.32	110.32 $\pm$ 36.03	+0.74	3.55×10^{-1}	
1.75	107.83 $\pm$ 27.17	108.25 $\pm$ 27.98	+0.42	3.60×10^{-1}	
2.00	101.84 $\pm$ 22.45	101.72 $\pm$ 22.91	−0.12	7.50×10^{-1}	
2.25	95.75 $\pm$ 18.77	95.14 $\pm$ 19.53	−0.60	2.63×10^{-2}	*
2.50	90.68 $\pm$ 14.40	90.31 $\pm$ 14.92	−0.37	2.13×10^{-1}	
2.75	85.47 $\pm$ 11.80	85.50 $\pm$ 12.20	+0.02	9.31×10^{-1}	
3.00	81.00 $\pm$ 9.60	80.75 $\pm$ 10.02	−0.25	3.05×10^{-1}	
3.25	77.03 $\pm$ 7.49	76.67 $\pm$ 8.00	−0.36	1.22×10^{-1}	
3.50	73.08 $\pm$ 6.34	72.78 $\pm$ 6.94	−0.31	1.45×10^{-1}	

B Python code

```

1  # %% [markdown]
2  # ## Packages
3
4  # %%
5  # Standard library
6  import os
7  import re
8  import time
9  import glob
10 import math
11
12 # Science
13 import numpy as np
14 import pandas as pd
15 import scipy.io as sio
16 from scipy.io import savemat
17 from scipy import signal
18 from scipy.stats import ttest_rel
19
20 # Plotting
21 import matplotlib.pyplot as plt
22 import matplotlib as mpl
23
24 # Scikit learn
25 from sklearn.cross_decomposition import CCA
26 from sklearn.metrics import confusion_matrix
27 from sklearn.pipeline import Pipeline
28 from sklearn.preprocessing import StandardScaler
29 from sklearn.svm import SVC, LinearSVC
30
31 # %% [markdown]
32 # ## Variables and small file import
33
34 # %%
35 # 9 Channels chosen to fit active occipital/parietal region:
36 # Pz, PO5, PO3, POz, PO4, PO6, O1, Oz, O2
37 channel_labels = ['Pz', 'PO5', 'PO3', 'POz', 'PO4', 'PO6', 'O1', 'Oz', 'O2']
38 channels = [47, 53, 54, 55, 56, 57, 60, 61, 62]
39
40 label_to_idx = {lab: i for i, lab in enumerate(channel_labels)}
41 fs = 250
42 sub_n = 35
43
44 #paths
45 base_path = r'\SSVEP-BCI-Data'
46 label_path = base_path + r'\Freq_Phase.mat'
47 meta_path = base_path + r'\subject_info_35_dataSets.txt'
48
49 # load labels from trials
50 labels_fr = sio.loadmat(label_path)['freqs']
51 labels_ph = sio.loadmat(label_path)['phases']
52
53 # load meta data
54 metadata = open(meta_path)
55
56 # %% [markdown]
57 # ## Batch Import and channel selection
58
59 # %%
60 # Iterate over the data of 35 subjects

```

```

61 subdata_channels = []
62
63 for sub in range(sub_n):
64     subdata_path = base_path + r'\S' + str(sub + 1) + r'.mat'
65
66     # subdata shape: [n_channels, n_samples, n_trials, n_trial_blocks]
67     subdata = sio.loadmat(subdata_path)['data']
68
69     # extract the channels of interest
70     sub_ch = subdata[channels, :, :, :]
71
72     subdata_channels.append(sub_ch)
73
74     # Print the subject number and the shape of EEG matrices for progress
75     print('Subject ' + str(sub + 1))
76     print(sub_ch.shape)
77
78 # %% [markdown]
79 # ## Pre-processing (filtering) module
80
81 # %%
82 nB, nA = signal.iircomb(50, 35, ftype='notch', fs=fs)
83
84 # Iterate over the data of 35 subjects
85 filtered_data = []
86
87 for sub in range(sub_n):
88     subdata_raw = subdata_channels[sub]
89
90     n_data = signal.filtfilt(nB, nA, subdata_raw, axis = 1, padtype='odd', padlen=3*(max(len(nB),
91     len(nA))-1))
92     filtered_data.append(n_data)
93
94     # Print the subject number and the shape of EEG matrices for progress
95     print('Subject ' + str(sub + 1) + ' filtered')
96
97 # %% [markdown]
98 # ## Feature extraction
99
100 # %%
101 def cca_reference(list_freqs, fs, n_smpls, n_harms=3):
102     """
103     y_ref[k] = reference sinusoids
104     Output: (K, 2H, T)
105     """
106     f = np.asarray(list_freqs, dtype=float).reshape(-1) # (K,)
107     t = np.arange(n_smpls, dtype=float) / fs # (T,)
108     h = np.arange(1, n_harms + 1, dtype=float) # (H,)
109
110     ang = 2.0 * np.pi * f[:, None, None] * h[None, :, None] * t[None, None, :] # (K,H,T)
111
112     y_ref = np.empty((f.size, 2 * n_harms, n_smpls), dtype=float)
113     y_ref[:, 0::2, :] = np.sin(ang)
114     y_ref[:, 1::2, :] = np.cos(ang)
115     return y_ref
116
117 def filterbank_sos(fs, num_fbs=7, f_high=90.0, step_hz=8.0,
118     low_trans_hz=2.0, high_trans_hz=10.0,
119     gpass=3, gstop=40, rp=0.5):
120     """
121     Chen 2015: subband k has passband [step_hz*k, f_high]
122     with stopband [step_hz*k - low_trans_hz, f_high + high_trans_hz].

```

```

122     Returns a list of SOS filters.
123     """
124     nyq = fs / 2.0
125     f_high = min(float(f_high), 0.98 * nyq)
126     f_stop_high = min(f_high + float(high_trans_hz), 0.98 * nyq)
127
128     sos_list = []
129
130     for k in range(1, num_fbs + 1):
131         wp_low = step_hz * k
132         ws_low = max(0.1, wp_low - low_trans_hz)
133
134         # Keep within valid range
135         wp_low = min(wp_low, 0.98 * nyq)
136         ws_low = min(ws_low, 0.98 * nyq)
137
138         N, Wn = signal.cheblord([wp_low, f_high], [ws_low, f_stop_high],
139                                gpass=gpass, gstop=gstop, fs=fs)
140         sos = signal.cheb1(N, rp=rp, Wn=Wn, btype="bandpass", fs=fs, output="sos")
141         sos_list.append(sos)
142
143     return sos_list
144
145 def fbcca_scores(epoch, y_ref, sos_list, fb_a=1.25, fb_b=0.25):
146     """
147     epoch: (n_ch, T)
148     y_ref: (K, 2H, T)
149     returns: rho (K,)
150     """
151     X0 = np.asarray(epoch, dtype=float)
152     K = y_ref.shape[0]
153     num_fbs = len(sos_list)
154
155     T = X0.shape[1]
156     fb_coefs = (np.arange(1, num_fbs + 1, dtype=float) ** (-fb_a)) + fb_b
157     r2 = np.zeros((num_fbs, K), dtype=float)
158
159     for fb_i, sos in enumerate(sos_list):
160         try:
161             Xf = signal.sosfiltfilt(sos, X0, axis=1).T # (T, n_ch)
162         except ValueError:
163             safe_padlen = max(0, T - 1)
164             Xf = signal.sosfiltfilt(sos, X0, axis=1, padlen=safe_padlen).T
165
166         for k in range(K):
167             Y = y_ref[k].T # (T, 2H)
168             cca = CCA(n_components=1, max_iter=1000)
169             Xc, Yc = cca.fit_transform(Xf, Y)
170             rr = np.corrcoef(Xc[:, 0], Yc[:, 0])[0, 1]
171             r2[fb_i, k] = 0.0 if not np.isfinite(rr) else rr * rr
172
173     rho = fb_coefs @ r2
174     return rho
175
176
177 def build_epoch_length_indices(fs, t_cue=0.5, t_latency=0.14, t_gaze=0.5):
178     start = int(round((t_cue + t_latency) * fs))
179     stop = start + int(round(t_gaze * fs))
180     return np.arange(start, stop, dtype=int)
181
182
183 def fbcca_features_per_subject(sub_data, epoch_idx, y_ref, sos_list, ch_label=None):

```

```

184     """
185     Extract FBCCA score vectors for one subject.
186
187     Inputs
188     sub_data: (n_ch, n_samples, n_trials, n_blocks)
189     epoch_idx: sample indices for the time window
190     y_ref: (K, 2*n_harms, T)
191     sos_list: list of SOS filters
192     ch_label:
193         None          -> use all loaded channels
194         string        -> use one channel by label
195
196     Output
197     feats: (n_trials, n_blocks, K)
198     """
199     if ch_label is None:
200         x = sub_data
201     else:
202         if ch_label not in label_to_idx:
203             raise ValueError(f"Unknown channel label: {ch_label}")
204         ch_idx = label_to_idx[ch_label]
205         x = sub_data[ch_idx, :, :, :] # keep channel dimension
206
207     _, _, n_trials, n_blocks = x.shape
208     K = y_ref.shape[0]
209
210     feats = np.zeros((n_trials, n_blocks, K), dtype=float)
211
212     for tr in range(n_trials):
213         for bl in range(n_blocks):
214             epoch = x[:, epoch_idx, tr, bl]
215             feats[tr, bl] = fbcca_scores(epoch, y_ref, sos_list)
216
217     return feats
218
219 def phase_estimation(eeg_segment, target_frequencies, predicted_idx):
220     fs = 250
221
222     if isinstance(predicted_idx, (int, np.integer)):
223         predicted_idx = np.array([predicted_idx])
224         return_scalar = True
225     else:
226         predicted_idx = np.array(predicted_idx)
227         return_scalar = False
228
229     n_fft = 10000
230     f_data = np.fft.fft(eeg_segment, n=n_fft)
231     f_data_shifted = np.squeeze(np.fft.fftshift(f_data))
232     f_axis = np.linspace(-fs/2, fs/2, n_fft, endpoint=False)
233     phases = np.zeros(len(predicted_idx), dtype=np.float64)
234
235     # Process each requested frequency index
236     for i, freq_idx in enumerate(predicted_idx):
237         target_freq = target_frequencies[freq_idx]
238         freq_mask = (f_axis > target_freq - 0.05) & (f_axis < target_freq + 0.05)
239         f_data_in_band = f_data_shifted[freq_mask]
240
241         # Find index of maximum magnitude
242         max_idx = np.argmax(np.abs(f_data_in_band))
243         freq_bin_indices = np.where(freq_mask)[0]
244
245         # Get phase at the peak

```

```

246         phases[i] = np.angle(f_data_shifted[freq_bin_indices[max_idx]])
247
248     # Return scalar if input was scalar
249     if return_scalar:
250         return phases[0]
251     return phases
252
253
254 def fbcca_phase_features_per_subject(sub_data, epoch_idx, y_ref, sos_list,
255                                     ch_label, channel_labels, target_freqs, fbcca_preds, fs):
256     """
257     sub_data: (n_ch, n_samples, n_trials, n_blocks) for one subject
258     Returns: feats (n_trials, n_blocks, 40 + 2)
259     """
260     label_to_idx = {lab: i for i, lab in enumerate(channel_labels)}
261     K = y_ref.shape[0]
262     n_pred = int(fbcca_preds)
263
264     # Decide which channels to use
265     if ch_label is None:
266         x = sub_data
267         phase_ch_in_x = 0
268     else:
269         if ch_label not in label_to_idx:
270             raise ValueError(f"Unknown channel label: {ch_label}")
271         ch_idx = label_to_idx[ch_label]
272         x = sub_data[ch_idx, :, :, :]
273         phase_ch_in_x = 0
274
275     _, _, n_trials, n_blocks = x.shape
276     feats = np.zeros((n_trials, n_blocks, K + 2 * n_pred), dtype=float)
277
278     for tr in range(n_trials):
279         for bl in range(n_blocks):
280             epoch = x[:, epoch_idx, tr, bl] # (n_ch_used, T)
281
282             # FBCCA scores
283             rho = fbcca_scores(epoch, y_ref, sos_list) # (K,)
284             feats[tr, bl, :K] = rho
285
286             pred_idx = np.argsort(rho)[::-1][:n_pred]
287
288             eeg_1ch = epoch[phase_ch_in_x, :] # (T,)
289             ang = phase_estimation(eeg_1ch, target_freqs, pred_idx) # (n_pred,)
290
291             feats[tr, bl, K:K + 2 * n_pred:2] = np.cos(ang)
292             feats[tr, bl, K + 1:K + 2 * n_pred:2] = np.sin(ang)
293
294     return feats
295
296
297 # %% [markdown]
298 # ### Feature extraction for both FBCCA and phase
299
300 # %%
301 # Parameters
302 num_harms = 5 # 2015 FBCCA Chen supports this for accuracy
303 num_fbs = 7 # 2015 FBCCA Chen supports this for accuracy
304 f_high = 90.0 # 2015 FBCCA Chen supports this for the optimal bound for harmonics
305
306 t_cue = 0.5 # Value given by the Benchmark dataset
307 t_latency = 0.14 # Benchmark dataset supports this value for average gaze latency across the

```

```

    dataset
308 t_w = 2 #2015 FBCCA Chen supports this for accuracy and ITR
309
310 # Target frequencies for phase
311 target_freqs = np.asarray(labels_fr).reshape(-1)
312
313 # Indices for the epoch inside each 6 s trial
314 epoch_idx = build_epoch_length_indices(
315     fs=fs,
316     t_cue=t_cue,
317     t_latency=t_latency,
318     t_gaze=t_w
319 )
320
321 # Stimulation freqs
322 list_freqs = np.asarray(labels_fr).reshape(-1)
323
324 # CCA reference frequencies from the known stimulation frequencies
325 y_ref = cca_reference(
326     list_freqs=list_freqs,
327     fs=fs,
328     n_smpls=len(epoch_idx),
329     n_harms=num_harms
330 )
331
332 # Filterbank with number of filters
333 sos_list = filterbank_sos(
334     fs=fs,
335     num_fbs=num_fbs,
336     f_high=f_high
337 )
338
339 information = np.zeros((len(channel_labels),2))
340 print(information.shape)
341
342 fbcca_preds = 3
343
344 all_feats_by_channel = [] # shape (sub_n, 40, 6, n_features)
345
346 for i, chosen_channel in enumerate(channel_labels):
347     fbcca_phase_feats_ch = []
348     print(chosen_channel)
349
350     for sub in range(sub_n):
351         fbcca_phase_feats_sub = fbcca_phase_features_per_subject(
352             sub_data=filtered_data[sub],
353             epoch_idx=epoch_idx,
354             y_ref=y_ref,
355             sos_list=sos_list,
356             ch_label=chosen_channel,
357             channel_labels=channel_labels,
358             target_freqs=target_freqs,
359             fbcca_preds = fbcca_preds,
360             fs=fs
361         )
362         fbcca_phase_feats_ch.append(fbcca_phase_feats_sub)
363         print("Subject", sub + 1, "channel", chosen_channel, "done")
364
365     fbcca_feats_ch = np.stack(fbcca_phase_feats_ch, axis=0)
366     print("FBCCA features shape:", fbcca_feats_ch.shape)
367
368     all_feats_by_channel.append(fbcca_feats_ch.astype(np.float32))

```



```

369
370     # fbcca_feats_list element shape: (40, 6, 40)
371     n_subjects, n_trials, n_blocks, n_targets = fbcca_feats_ch.shape
372
373     # Timing for ITR
374     tw = t_w
375     t_break = 0.5
376     t_latency = 0.14
377     t_comp = 0.0
378
379     total_t = tw + t_break + t_latency + t_comp
380     N = n_trials
381
382     # LOSO evaluation
383     cm_total = np.zeros((n_trials, n_trials), dtype=int)
384     acc_folds = []
385     itr_folds = []
386
387     for test_sub in range(sub_n):
388         train_subs = [s for s in range(sub_n) if s != test_sub]
389
390         # Training set: all epochs from all training subjects
391         # shape before reshape: (train_subjects, 40, 6, 40)
392         X_train = fbcca_feats_ch[train_subs].reshape(len(train_subs) * n_trials * n_blocks,
n_targets)
393
394         # Labels: for each subject, trials 0..39 repeated for each block
395         y_train_one_sub = np.repeat(np.arange(n_trials), n_blocks) # length = 40*6
396         y_train = np.tile(y_train_one_sub, len(train_subs)) # repeat for each training
subject
397
398         # Test set: all epochs from held out subject
399         X_test = fbcca_feats_ch[test_sub].reshape(n_trials * n_blocks, n_targets)
400         y_test = np.repeat(np.arange(n_trials), n_blocks)
401
402         pipe = Pipeline([
403             ("scaler", StandardScaler()),
404             ("svc", SVC(kernel="rbf", C=1, gamma=0.01))
405         ])
406
407         pipe.fit(X_train, y_train)
408         y_pred = pipe.predict(X_test)
409
410         # Confusion matrix
411         cm_fold = confusion_matrix(y_test, y_pred, labels=np.arange(n_trials))
412         cm_total += cm_fold
413
414         # Accuracy for this subject fold
415         acc = np.mean(y_pred == y_test)
416         acc_folds.append(acc)
417
418         # ITR for this fold
419         if acc == 1:
420             itr = 60 / total_t * np.log2(N)
421         elif acc < 1 / N:
422             itr = 0.0
423         else:
424             itr = 60 / total_t * (
425                 np.log2(N)
426                 + acc * np.log2(acc)
427                 + (1 - acc) * np.log2((1 - acc) / (N - 1))
428             )

```

```

429         itr_folds.append(float(itr))
430
431     acc_mean = float(np.mean(acc_folds))
432     itr_mean = float(np.mean(itr_folds))
433
434     print("Mean accuracy:", acc_mean)
435     print("Mean ITR (bits/min):", itr_mean)
436     print("Confusion matrix shape:", cm_total.shape)
437
438     savemat(
439         "fbcca_phase_ch_features.mat",
440         {
441             "features": fbcca_phase_feats_ch,      # numpy array
442             "channel_labels": channel_labels,      # optional metadata
443             "fs": fs,
444             "epoch_idx": epoch_idx,
445             "target_freqs": target_freqs,
446         }
447     )
448
449     # %% [markdown]
450     # ### Grid search
451
452     # %%
453     tw_seq = np.arange(0.25, 3.50 + 1e-12, 0.25)
454
455     num_harms = 5
456     num_fbs = 7
457     f_high = 90.0
458     fbcca_preds = 3
459
460     t_cue = 0.5
461     t_latency = 0.14
462     t_break = 0.5
463     t_comp = 0.0
464
465     svc_params = dict(kernel="rbf", C=1, gamma=0.01)
466
467     # Setups to evaluate
468     setups = [
469         ("All9", None),
470         ("Oz", "Oz")
471     ]
472
473     # Output folder for this grid search
474     grid_dir = r"\TW_Grid"
475     os.makedirs(grid_dir, exist_ok=True)
476
477     target_freqs = np.asarray(labels_fr).reshape(-1)
478     list_freqs = np.asarray(labels_fr).reshape(-1)
479
480     def loso_eval_subjectwise(X, svc_params, tw, t_break, t_latency, t_comp):
481         """
482         X: (n_subjects, n_trials, n_blocks, n_features)
483         Returns per-subject accuracy and ITR arrays
484         """
485         n_subjects, n_trials, n_blocks, n_feats = X.shape
486         total_t = float(tw) + float(t_break) + float(t_latency) + float(t_comp)
487         N = int(n_trials)
488
489         y_one_sub = np.repeat(np.arange(n_trials), n_blocks)
490

```

```

491 acc_sub = np.zeros(n_subjects, dtype=float)
492 itr_sub = np.zeros(n_subjects, dtype=float)
493
494 for test_sub in range(n_subjects):
495     train_subs = [s for s in range(n_subjects) if s != test_sub]
496
497     X_train = X[train_subs].reshape(len(train_subs) * n_trials * n_blocks, n_feats)
498     y_train = np.tile(y_one_sub, len(train_subs))
499
500     X_test = X[test_sub].reshape(n_trials * n_blocks, n_feats)
501     y_test = y_one_sub
502
503     pipe = Pipeline([
504         ("scaler", StandardScaler()),
505         ("svc", SVC(**svc_params)),
506     ])
507
508     pipe.fit(X_train, y_train)
509     y_pred = pipe.predict(X_test)
510
511     acc = float(np.mean(y_pred == y_test))
512     acc_sub[test_sub] = acc
513
514     if acc == 1.0:
515         itr = 60.0 / total_t * np.log2(N)
516     elif acc < 1.0 / N:
517         itr = 0.0
518     else:
519         itr = 60.0 / total_t * (
520             np.log2(N)
521             + acc * np.log2(acc)
522             + (1.0 - acc) * np.log2((1.0 - acc) / (N - 1))
523         )
524     itr_sub[test_sub] = float(itr)
525
526     return acc_sub, itr_sub
527
528 # Compute filterbank once
529 sos_list = filterbank_sos(fs=fs, num_fbs=num_fbs, f_high=f_high)
530
531 # Get number of samples per trial for bounds checking
532 n_samples_trial = int(filtered_data[0].shape[1])
533
534 # CSV summary
535 rows = []
536
537 # Run grid
538 for setup_name, ch_label in setups:
539     print("Setup:", setup_name, "| ch_label:", ch_label)
540
541     for t_w in tw_seq:
542         t_w = float(t_w)
543         print("\n--- t_w =", t_w, "s ---")
544
545         # Epoch indices and bounds check
546         epoch_idx = build_epoch_length_indices(fs=fs, t_cue=t_cue, t_latency=t_latency, t_gaze=
t_w)
547
548         # Reference signals depend on epoch length
549         y_ref = cca_reference(list_freqs=list_freqs, fs=fs, n_smpls=len(epoch_idx), n_harms=
num_harms)
550

```

```

551     # Feature extraction
552     feats_list = []
553     for sub in range(sub_n):
554         feats_sub = fbcca_phase_features_per_subject(
555             sub_data=filtered_data[sub],
556             epoch_idx=epoch_idx,
557             y_ref=y_ref,
558             sos_list=sos_list,
559             ch_label=ch_label,
560             channel_labels=channel_labels,
561             target_freqs=target_freqs,
562             fbcca_preds=fbcca_preds,
563             fs=fs
564         )
565         feats_list.append(feats_sub.astype(np.float32))
566
567     feats = np.stack(feats_list, axis=0) # (sub, trial, block, feat)
568
569     # LOSO evaluation
570     acc_sub, itr_sub = loso_eval_subjectwise(feats, svc_params, t_w, t_break, t_latency,
571     t_comp)
572
573     acc_mean = float(np.mean(acc_sub))
574     acc_sd = float(np.std(acc_sub, ddof=1))
575     itr_mean = float(np.mean(itr_sub))
576     itr_sd = float(np.std(itr_sub, ddof=1))
577
578     print(f"Accuracy: {acc_mean:.4f}  sd {acc_sd:.4f}")
579     print(f"ITR:      {itr_mean:.4f}  sd {itr_sd:.4f} bits/min")
580
581     # Save one MAT per variation
582     tw_str = f"{t_w:.2f}".replace(".", "p")
583     ch_str = "All9" if ch_label is None else ch_label
584     mat_name = f"FBCCA_PHASE_{ch_str}_tw{tw_str}.mat"
585     mat_path = os.path.join(grid_dir, mat_name)
586
587     savemat(
588         mat_path,
589         {
590             # Core data
591             "features": feats,
592             "acc_subject": acc_sub.astype(np.float32),
593             "itr_subject": itr_sub.astype(np.float32),
594
595             # Summary stats
596             "acc_mean": np.float32(acc_mean),
597             "acc_sd": np.float32(acc_sd),
598             "itr_mean": np.float32(itr_mean),
599             "itr_sd": np.float32(itr_sd),
600
601             # Metadata
602             "setup": setup_name,
603             "ch_label": "" if ch_label is None else ch_label,
604             "channel_labels": np.array(channel_labels, dtype=object),
605
606             # Timing and indexing
607             "fs": float(fs),
608             "t_w": float(t_w),
609             "t_cue": float(t_cue),
610             "t_latency": float(t_latency),
611             "t_break": float(t_break),
612             "t_comp": float(t_comp),

```

```

612         "epoch_idx": epoch_idx.astype(np.int32),
613
614         # FBCCA parameters
615         "target_freqs": target_freqs.astype(np.float32),
616         "num_harms": int(num_harms),
617         "num_fbs": int(num_fbs),
618         "f_high": float(f_high),
619         "fbcca_preds": int(fbcca_preds),
620
621         # Classifier config
622         "svc_params": str(svc_params),
623     }
624 )
625
626 print("Saved:", mat_path)
627
628 rows.append([setup_name, ch_str, t_w, acc_mean, acc_sd, itr_mean, itr_sd, mat_path])
629
630 # Save CSV summary
631 df = pd.DataFrame(
632     rows,
633     columns=["setup", "channel_rule", "t_w", "acc_mean", "acc_sd", "itr_mean", "itr_sd", "
634             mat_file"]
635 )
636 csv_path = os.path.join(grid_dir, "tw_grid_results.csv")
637 df.to_csv(csv_path, index=False)
638 print("\nSaved CSV:", csv_path)
639
640 df.head()
641
642 # %% [markdown]
643 # ## Classification and statistics
644
645 # %% [markdown]
646 # ### Single-channel
647
648 # %%
649 # %% Paths and constants
650 project_root = r"\Project"
651
652 features_old_dir = os.path.join(project_root, "Scripts", "Single Channel With Phase")
653 features_new_dir = os.path.join(features_old_dir, "New Phase Parameters")
654
655 out_dir = r"plots"
656 os.makedirs(out_dir, exist_ok=True)
657
658 fs = 250
659 sub_n = 35
660 n_trials = 40
661 n_blocks = 6
662
663 channel_labels = ['Pz', 'PO5', 'PO3', 'POz', 'PO4', 'PO6', 'O1', 'Oz', 'O2']
664
665 t_w = 2
666 t_break = 0.5
667 t_latency = 0.14
668 t_comp = 0.0
669 total_t = t_w + t_break + t_latency + t_comp
670
671 # %%
672 FIG_W = 12
673 FIG_H = 5

```

```

673
674 mpl.rcParams.update({
675     "font.family": "Arial",
676     "font.size": 16,
677     "axes.titlesize": 16,
678     "axes.labelsize": 16,
679     "xtick.labelsize": 14,
680     "ytick.labelsize": 14,
681     "legend.fontsize": 12,
682     "lines.linewidth": 1.0,
683     "lines.markersize": 5.0,
684     "figure.figsize": (FIG_W, FIG_H),
685     "figure.dpi": 200,
686     "savefig.dpi": 300,
687     "savefig.bbox": "tight",
688     "savefig.pad_inches": 0.02,
689     "axes.spines.top": False,
690     "axes.spines.right": False,
691 })
692
693 # %%
694 def load_first_array_from_mat(mat_path, prefer_key="features"):
695     d = sio.loadmat(mat_path)
696     if prefer_key in d and isinstance(d[prefer_key], np.ndarray):
697         return d[prefer_key]
698     for k, v in d.items():
699         if k.startswith("__"):
700             continue
701         if isinstance(v, np.ndarray):
702             return v
703     raise ValueError(f"No ndarray found in {mat_path}")
704
705 def infer_channel_suffix(path):
706     base = os.path.splitext(os.path.basename(path))[0]
707     parts = base.split("_")
708     return parts[-1]
709
710 def itr_from_acc(acc, N, total_t):
711     acc = float(acc)
712     if acc == 1.0:
713         return float(60.0 / total_t * np.log2(N))
714     if acc < 1.0 / N:
715         return 0.0
716     return float(
717         60.0 / total_t * (
718             np.log2(N)
719             + acc * np.log2(acc)
720             + (1 - acc) * np.log2((1 - acc) / (N - 1))
721         )
722     )
723
724 def star_from_p(p):
725     if not np.isfinite(p):
726         return ""
727     if p < 1e-4:
728         return "****"
729     if p < 1e-3:
730         return "***"
731     if p < 1e-2:
732         return "**"
733     if p < 5e-2:
734         return "*"

```

```

735     return ""
736
737 def draw_vertical_stars(ax, x, p_value, y_top, y_span, color="k"):
738     if not np.isfinite(p_value):
739         return
740     if p_value < 1e-4:
741         n = 4
742     elif p_value < 1e-3:
743         n = 3
744     elif p_value < 1e-2:
745         n = 2
746     elif p_value < 5e-2:
747         n = 1
748     else:
749         return
750
751     dy = 0.03 * y_span
752     for i in range(n):
753         ax.text(
754             x,
755             y_top - i * dy,
756             "*",
757             ha="center",
758             va="top",
759             color=color,
760             fontsize=12,
761             fontweight="bold"
762         )
763
764 def save_plot_data_csv(
765     csv_path,
766     plot_id,
767     panel,
768     x_values,
769     series_dict,
770     unit,
771     p_values,
772     stars,
773 ):
774     x_values = list(x_values)
775     n = len(x_values)
776     x_index = np.arange(n, dtype=int)
777
778     if len(p_values) != n:
779         raise ValueError("p_values must have same length as x_values")
780     if len(stars) != n:
781         raise ValueError("stars must have same length as x_values")
782
783     rows = []
784     for series, (y, yerr) in series_dict.items():
785         y = np.asarray(y).reshape(-1)
786         yerr = np.asarray(yerr).reshape(-1)
787         if len(y) != n or len(yerr) != n:
788             raise ValueError(f"Length mismatch in series '{series}'")
789
790         for i in range(n):
791             rows.append({
792                 "plot_id": plot_id,
793                 "panel": panel,
794                 "series": series,
795                 "x_index": int(x_index[i]),
796                 "x_value": x_values[i],

```

```

797         "y": float(y[i]),
798         "yerr": float(yerr[i]),
799         "unit": unit,
800         "p_value": float(p_values[i]) if np.isfinite(p_values[i]) else np.nan,
801         "stars": str(stars[i]),
802     })
803
804     df = pd.DataFrame(rows)
805     write_header = not os.path.exists(csv_path)
806     df.to_csv(csv_path, mode="a", header=write_header, index=False)
807     print(f"Saved plot data ({panel}) -> {csv_path}")
808     return df
809
810
811 # %% [markdown]
812 # ##### Classifier parameters
813
814 # %%
815 svc_params = dict(kernel="rbf", C=1, gamma=0.01)
816
817 use_linear = False
818 lin_params = dict(C=1.0)
819
820 def make_clf():
821     if use_linear:
822         return Pipeline([
823             ("scaler", StandardScaler()),
824             ("clf", LinearSVC(**lin_params))
825         ])
826     return Pipeline([
827         ("scaler", StandardScaler()),
828         ("clf", SVC(**svc_params))
829     ])
830
831
832 # %% [markdown]
833 # ##### Discover feature files
834
835 # %%
836
837 fbcca_files = sorted(glob.glob(os.path.join(features_old_dir, "FBCCA_Feats_*")))
838 phase_files = sorted(glob.glob(os.path.join(features_new_dir, "Phase_Feats_*")))
839
840 fbcca_by_ch = {infer_channel_suffix(p): p for p in fbcca_files}
841 phase_by_ch = {infer_channel_suffix(p): p for p in phase_files}
842
843 channels_available = [ch for ch in channel_labels if (ch in fbcca_by_ch and ch in phase_by_ch)]
844 print("Channels available:", channels_available)
845
846
847 # %%
848 def loso_eval_subjectwise_blockstats(feats, make_clf, n_trials, n_blocks, total_t):
849     """
850     feats: (sub, trial, block, feat)
851
852     Returns:
853     acc_mean_sub, acc_sd_sub, itr_mean_sub, itr_sd_sub, acc_blocks_sub, itr_blocks_sub
854     where acc_blocks_sub and itr_blocks_sub have shape (sub, n_blocks)
855     """
856     feats = np.asarray(feats)
857     n_subjects, nT, nB, nF = feats.shape
858     if (nT, nB) != (n_trials, n_blocks):

```



```

859         raise ValueError(f"Unexpected feats shape {feats.shape}")
860
861     y_one_sub = np.repeat(np.arange(n_trials), n_blocks)
862
863     acc_mean = np.zeros(n_subjects, dtype=float)
864     acc_sd = np.zeros(n_subjects, dtype=float)
865     itr_mean = np.zeros(n_subjects, dtype=float)
866     itr_sd = np.zeros(n_subjects, dtype=float)
867
868     acc_blocks_all = np.zeros((n_subjects, n_blocks), dtype=float)
869     itr_blocks_all = np.zeros((n_subjects, n_blocks), dtype=float)
870
871     for test_sub in range(n_subjects):
872         train_subs = [s for s in range(n_subjects) if s != test_sub]
873
874         X_train = feats[train_subs].reshape(len(train_subs) * n_trials * n_blocks, nF)
875         y_train = np.tile(y_one_sub, len(train_subs))
876
877         X_test = feats[test_sub].reshape(n_trials * n_blocks, nF)
878         y_test = y_one_sub
879
880         clf = make_clf()
881         clf.fit(X_train, y_train)
882         y_pred = clf.predict(X_test)
883
884         y_pred_blk = y_pred.reshape(n_trials, n_blocks)
885         y_true_blk = np.arange(n_trials)[:n_blocks] * np.ones((1, n_blocks), dtype=int)
886
887         acc_blk = np.mean(y_pred_blk == y_true_blk, axis=0)
888         itr_blk = np.array([itr_from_acc(a, N=n_trials, total_t=total_t) for a in acc_blk], dtype
                             =float)
889
890         acc_blocks_all[test_sub, :] = acc_blk
891         itr_blocks_all[test_sub, :] = itr_blk
892
893         acc_mean[test_sub] = float(np.mean(acc_blk))
894         acc_sd[test_sub] = float(np.std(acc_blk, ddof=1)) if n_blocks > 1 else 0.0
895         itr_mean[test_sub] = float(np.mean(itr_blk))
896         itr_sd[test_sub] = float(np.std(itr_blk, ddof=1)) if n_blocks > 1 else 0.0
897
898     return acc_mean, acc_sd, itr_mean, itr_sd, acc_blocks_all, itr_blocks_all
899
900
901 # %% [markdown]
902 # #### Reading and classifying feature tensors
903
904 # %%
905 rows = []
906 per_subject_rows = []
907
908 cache = {}
909
910 for ch_i, ch in enumerate(channels_available, 1):
911     print(f"Evaluating channel {ch} ({ch_i}/{len(channels_available)})")
912
913     FB = load_first_array_from_mat(fbcca_by_ch[ch]).astype(np.float32)
914     PH = load_first_array_from_mat(phase_by_ch[ch]).astype(np.float32)
915
916     acc_fb_m, acc_fb_sd, itr_fb_m, itr_fb_sd, acc_fb_blk, itr_fb_blk =
917         loso_eval_subjectwise_blockstats(
918             FB, make_clf, n_trials=n_trials, n_blocks=n_blocks, total_t=total_t
919         )

```

```

919     acc_ph_m, acc_ph_sd, itr_ph_m, itr_ph_sd, acc_ph_blk, itr_ph_blk =
        loso_eval_subjectwise_blockstats(
920         PH, make_clf, n_trials=n_trials, n_blocks=n_blocks, total_t=total_t
921     )
922
923     cache[ch] = {
924         "acc_fb_m": acc_fb_m, "acc_fb_sd": acc_fb_sd, "itr_fb_m": itr_fb_m, "itr_fb_sd":
        itr_fb_sd,
925         "acc_ph_m": acc_ph_m, "acc_ph_sd": acc_ph_sd, "itr_ph_m": itr_ph_m, "itr_ph_sd":
        itr_ph_sd,
926         "acc_fb_blk": acc_fb_blk, "itr_fb_blk": itr_fb_blk,
927         "acc_ph_blk": acc_ph_blk, "itr_ph_blk": itr_ph_blk,
928     }
929
930     for s in range(sub_n):
931         per_subject_rows.append({
932             "channel": ch,
933             "subject": s + 1,
934             "acc_fbcca_mean": acc_fb_m[s],
935             "acc_fbcca_sd": acc_fb_sd[s],
936             "itr_fbcca_mean": itr_fb_m[s],
937             "itr_fbcca_sd": itr_fb_sd[s],
938             "acc_phase_mean": acc_ph_m[s],
939             "acc_phase_sd": acc_ph_sd[s],
940             "itr_phase_mean": itr_ph_m[s],
941             "itr_phase_sd": itr_ph_sd[s],
942             "acc_diff": acc_ph_m[s] - acc_fb_m[s],
943             "itr_diff": itr_ph_m[s] - itr_fb_m[s],
944         })
945
946     p_acc = float(ttest_rel(acc_ph_m, acc_fb_m, nan_policy="omit").pvalue)
947     p_itr = float(ttest_rel(itr_ph_m, itr_fb_m, nan_policy="omit").pvalue)
948
949     rows.append({
950         "channel": ch,
951         "acc_fbcca_mean": float(np.mean(acc_fb_m)),
952         "acc_fbcca_sd": float(np.std(acc_fb_m, ddof=1)),
953         "acc_phase_mean": float(np.mean(acc_ph_m)),
954         "acc_phase_sd": float(np.std(acc_ph_m, ddof=1)),
955         "acc_pvalue": p_acc,
956         "acc_sig": star_from_p(p_acc),
957         "itr_fbcca_mean": float(np.mean(itr_fb_m)),
958         "itr_fbcca_sd": float(np.std(itr_fb_m, ddof=1)),
959         "itr_phase_mean": float(np.mean(itr_ph_m)),
960         "itr_phase_sd": float(np.std(itr_ph_m, ddof=1)),
961         "itr_pvalue": p_itr,
962         "itr_sig": star_from_p(p_itr),
963     })
964
965     summary_df = pd.DataFrame(rows).sort_values("acc_phase_mean", ascending=False).reset_index(drop=
        True)
966     per_sub_df = pd.DataFrame(per_subject_rows).sort_values(["channel", "subject"]).reset_index(drop=
        True)
967
968     summary_df.to_csv(os.path.join(out_dir, "channel_ranking_with_tests.csv"), index=False)
969     per_sub_df.to_csv(os.path.join(out_dir, "per_subject_all_channels.csv"), index=False)
970
971     print("Saved:", os.path.join(out_dir, "channel_ranking_with_tests.csv"))
972     print("Saved:", os.path.join(out_dir, "per_subject_all_channels.csv"))
973
974     summary_df.head()
975

```

```

976
977 # %% [markdown]
978 # ##### Plot 1: Channel ranking with per channel p values
979
980 # %%
981 plot_df = summary_df.copy()
982 x = np.arange(len(plot_df))
983 ch_names = plot_df["channel"].tolist()
984
985 fig, ax = plt.subplots(1, 2, figsize=(FIG_W, FIG_H), constrained_layout=True)
986
987 # Accuracy
988 ax0 = ax[0]
989 ax0.errorbar(x, plot_df["acc_fbcca_mean"].values * 100, yerr=plot_df["acc_fbcca_sd"].values *
1000             100,
990             fmt="o-", color='blue', label="FBCCA", capsize=5)
991 ax0.errorbar(x, plot_df["acc_phase_mean"].values * 100, yerr=plot_df["acc_phase_sd"].values *
1002             100,
992             fmt="o-", color='red', label="FBCCA + Phase", capsize=5)
993 ax0.set_xticks(x)
994 ax0.set_xticklabels(ch_names, rotation=45)
995 ax0.set_xlabel("Channel")
996 ax0.set_ylabel("Accuracy (%)")
997 ax0.set_ylim(-10, 130)
998 ax0.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
999
1000 y_span = ax0.get_ylim()[1] - ax0.get_ylim()[0]
1001 y_top = ax0.get_ylim()[1] - 0.01 * y_span
1002 for i in range(len(x)):
1003     draw_vertical_stars(ax0, i, float(plot_df["acc_pvalue"].iloc[i]), y_top, y_span)
1004
1005 # ITR
1006 ax1 = ax[1]
1007 ax1.errorbar(x, plot_df["itr_fbcca_mean"].values, yerr=plot_df["itr_fbcca_sd"].values,
1008             fmt="o-", color='blue', label="FBCCA", capsize=5)
1009 ax1.errorbar(x, plot_df["itr_phase_mean"].values, yerr=plot_df["itr_phase_sd"].values,
1010             fmt="o-", color='red', label="FBCCA + Phase", capsize=5)
1011 ax1.set_xticks(x)
1012 ax1.set_xticklabels(ch_names, rotation=45)
1013 ax1.set_xlabel("Channel")
1014 ax1.set_ylabel("ITR (bits/min)")
1015 ax1.set_ylim(-10, 170)
1016 ax1.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
1017
1018 y_span = ax1.get_ylim()[1] - ax1.get_ylim()[0]
1019 y_top = ax1.get_ylim()[1] - 0.01 * y_span
1020 for i in range(len(x)):
1021     draw_vertical_stars(ax1, i, float(plot_df["itr_pvalue"].iloc[i]), y_top, y_span)
1022
1023 # Export plot data CSV with per point p and stars
1024 csv_path = os.path.join(out_dir, "fig_channel_ranking.csv")
1025 if os.path.exists(csv_path):
1026     os.remove(csv_path)
1027
1028 save_plot_data_csv(
1029     csv_path=csv_path,
1030     plot_id="channel_ranking",
1031     panel="acc",
1032     x_values=ch_names,
1033     series_dict={
1034         "fbcca": (plot_df["acc_fbcca_mean"].values * 100, plot_df["acc_fbcca_sd"].values * 100),
1035         "phase": (plot_df["acc_phase_mean"].values * 100, plot_df["acc_phase_sd"].values * 100),

```

```

1036     },
1037     unit="percent",
1038     p_values=plot_df["acc_pvalue"].values,
1039     stars=plot_df["acc_sig"].values
1040 )
1041
1042 save_plot_data_csv(
1043     csv_path=csv_path,
1044     plot_id="channel_ranking",
1045     panel="itr",
1046     x_values=ch_names,
1047     series_dict={
1048         "fbcca": (plot_df["itr_fbcca_mean"].values, plot_df["itr_fbcca_sd"].values),
1049         "phase": (plot_df["itr_phase_mean"].values, plot_df["itr_phase_sd"].values),
1050     },
1051     unit="bits_per_min",
1052     p_values=plot_df["itr_pvalue"].values,
1053     stars=plot_df["itr_sig"].values
1054 )
1055
1056 plt.savefig(r"plots\Results_schannel.pdf")
1057 plt.show()
1058 print("Saved:", csv_path)
1059
1060
1061 # %% [markdown]
1062 # #### Plot 2: Best channel subjects, with per subject p values
1063
1064 # %%
1065
1066 best_ch = summary_df.iloc[0]["channel"]
1067 best_df = per_sub_df[per_sub_df["channel"] == best_ch].sort_values("subject").reset_index(drop=
    True)
1068
1069 acc_fb_blk = cache[best_ch]["acc_fb_blk"]
1070 acc_ph_blk = cache[best_ch]["acc_ph_blk"]
1071 itr_fb_blk = cache[best_ch]["itr_fb_blk"]
1072 itr_ph_blk = cache[best_ch]["itr_ph_blk"]
1073
1074 # Per subject p values across blocks
1075 p_acc_sub = np.array([ttest_rel(acc_ph_blk[s], acc_fb_blk[s], nan_policy="omit").pvalue for s in
    range(sub_n)], dtype=float)
1076 p_itr_sub = np.array([ttest_rel(itr_ph_blk[s], itr_fb_blk[s], nan_policy="omit").pvalue for s in
    range(sub_n)], dtype=float)
1077
1078 stars_acc_sub = np.array([star_from_p(p) for p in p_acc_sub], dtype=object)
1079 stars_itr_sub = np.array([star_from_p(p) for p in p_itr_sub], dtype=object)
1080
1081 x = best_df["subject"].values
1082
1083 fig, ax = plt.subplots(1, 2, figsize=(FIG_W, FIG_H), constrained_layout=True)
1084
1085 # Accuracy
1086 ax0 = ax[0]
1087 ax0.errorbar(x, best_df["acc_fbcca_mean"].values * 100, yerr=best_df["acc_fbcca_sd"].values *
    100,
1088             fmt="o", color='blue', label="FBCCA", capsizes=4)
1089 ax0.errorbar(x, best_df["acc_phase_mean"].values * 100, yerr=best_df["acc_phase_sd"].values *
    100,
1090             fmt="o", color='red', label="FBCCA + Phase", capsizes=4)
1091 ax0.set_xlabel("Subject")
1092 ax0.set_ylabel("Accuracy (%)")

```

```

1093 ax0.set_ylim(-10, 130)
1094 ax0.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
1095
1096 y_span = ax0.get_ylim()[1] - ax0.get_ylim()[0]
1097 y_top = ax0.get_ylim()[1] - 0.01 * y_span
1098 for i, subj in enumerate(x):
1099     draw_vertical_stars(ax0, subj, float(p_acc_sub[i]), y_top, y_span)
1100
1101 # ITR
1102 ax1 = ax[1]
1103 ax1.errorbar(x, best_df["itr_fbcca_mean"].values, yerr=best_df["itr_fbcca_sd"].values,
1104             fmt="o", color='blue', label="FBCCA", capsize=4)
1105 ax1.errorbar(x, best_df["itr_phase_mean"].values, yerr=best_df["itr_phase_sd"].values,
1106             fmt="o", color='red', label="FBCCA + Phase", capsize=4)
1107 ax1.set_xlabel("Subject")
1108 ax1.set_ylabel("ITR (bits/min)")
1109 ax1.set_ylim(-10, 170)
1110 ax1.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
1111
1112 y_span = ax1.get_ylim()[1] - ax1.get_ylim()[0]
1113 y_top = ax1.get_ylim()[1] - 0.01 * y_span
1114 for i, subj in enumerate(x):
1115     draw_vertical_stars(ax1, subj, float(p_itr_sub[i]), y_top, y_span)
1116
1117 # Export plot data CSV with per point p and stars
1118 csv_path = os.path.join(out_dir, f"fig_best_channel_{best_ch}_subjects.csv")
1119 if os.path.exists(csv_path):
1120     os.remove(csv_path)
1121
1122 save_plot_data_csv(
1123     csv_path=csv_path,
1124     plot_id=f"best_channel_{best_ch}_subjects",
1125     panel="acc",
1126     x_values=x,
1127     series_dict={
1128         "fbcca": (best_df["acc_fbcca_mean"].values * 100, best_df["acc_fbcca_sd"].values * 100),
1129         "phase": (best_df["acc_phase_mean"].values * 100, best_df["acc_phase_sd"].values * 100),
1130     },
1131     unit="percent",
1132     p_values=p_acc_sub,
1133     stars=stars_acc_sub
1134 )
1135
1136 save_plot_data_csv(
1137     csv_path=csv_path,
1138     plot_id=f"best_channel_{best_ch}_subjects",
1139     panel="itr",
1140     x_values=x,
1141     series_dict={
1142         "fbcca": (best_df["itr_fbcca_mean"].values, best_df["itr_fbcca_sd"].values),
1143         "phase": (best_df["itr_phase_mean"].values, best_df["itr_phase_sd"].values),
1144     },
1145     unit="bits_per_min",
1146     p_values=p_itr_sub,
1147     stars=stars_itr_sub
1148 )
1149
1150 plt.savefig(r"plots\Results_subject_sch.pdf")
1151 plt.show()
1152
1153 print("Saved:", csv_path)
1154

```

```

1155
1156 # %% [markdown]
1157 # ### 9-channel
1158
1159 # %%
1160 # %% Paths and constants
1161 project_root = r"\Project"
1162 features_dir = os.path.join(project_root, "Scripts", "Single Channel With Phase")
1163
1164 in_file = os.path.join(features_dir, "fbcca_phase_features.mat")
1165
1166 out_dir = os.path.join(project_root, "Scripts", "plots")
1167 os.makedirs(out_dir, exist_ok=True)
1168
1169 plots_dir = os.path.join(project_root, "Scripts", "plots")
1170 os.makedirs(plots_dir, exist_ok=True)
1171
1172 K = 40
1173 svc_params = dict(kernel="rbf", C=1, gamma=0.01)
1174
1175 t_w = 2.0
1176 t_break = 0.5
1177 t_latency = 0.14
1178 t_comp = 0.0
1179 total_t = t_w + t_break + t_latency + t_comp
1180
1181 print("Input:", in_file)
1182 print("Out:", out_dir)
1183 print("Plots:", plots_dir)
1184 print("Classifier:", svc_params)
1185 print("Timing total_t:", total_t)
1186
1187
1188 # %%
1189 def itr_from_acc(acc, N, total_t):
1190     acc = float(acc)
1191     if acc == 1.0:
1192         return float(60.0 / total_t * np.log2(N))
1193     if acc < 1.0 / N:
1194         return 0.0
1195     return float(
1196         60.0 / total_t * (
1197             np.log2(N)
1198             + acc * np.log2(acc)
1199             + (1 - acc) * np.log2((1 - acc) / (N - 1))
1200         )
1201     )
1202
1203 def star_from_p(p):
1204     if not np.isfinite(p):
1205         return ""
1206     if p < 1e-4:
1207         return "****"
1208     if p < 1e-3:
1209         return "***"
1210     if p < 1e-2:
1211         return "**"
1212     if p < 5e-2:
1213         return "*"
1214     return ""
1215
1216 def draw_vertical_stars(ax, x, p_value, y_top, y_span, color="k"):

```

```

1217     if not np.isfinite(p_value):
1218         return
1219     if p_value < 1e-4:
1220         n = 4
1221     elif p_value < 1e-3:
1222         n = 3
1223     elif p_value < 1e-2:
1224         n = 2
1225     elif p_value < 5e-2:
1226         n = 1
1227     else:
1228         return
1229
1230     dy = 0.03 * y_span
1231     for i in range(n):
1232         ax.text(
1233             x,
1234             y_top - i * dy,
1235             "*",
1236             ha="center",
1237             va="top",
1238             color=color,
1239             fontsize=12,
1240             fontweight="bold"
1241         )
1242
1243 def save_plot_data_csv(
1244     csv_path,
1245     plot_id,
1246     panel,
1247     x_values,
1248     series_dict,
1249     unit,
1250     p_values,
1251     stars,
1252 ):
1253     x_values = list(x_values)
1254     n = len(x_values)
1255     x_index = np.arange(n, dtype=int)
1256
1257     if len(p_values) != n:
1258         raise ValueError("p_values must have same length as x_values")
1259     if len(stars) != n:
1260         raise ValueError("stars must have same length as x_values")
1261
1262     rows = []
1263     for series, (y, yerr) in series_dict.items():
1264         y = np.asarray(y).reshape(-1)
1265         yerr = np.asarray(yerr).reshape(-1)
1266         if len(y) != n or len(yerr) != n:
1267             raise ValueError(f"Length mismatch in series '{series}'")
1268
1269         for i in range(n):
1270             rows.append({
1271                 "plot_id": plot_id,
1272                 "panel": panel,
1273                 "series": series,
1274                 "x_index": int(x_index[i]),
1275                 "x_value": x_values[i],
1276                 "y": float(y[i]),
1277                 "yerr": float(yerr[i]),
1278                 "unit": unit,

```

```

1279         "p_value": float(p_values[i]) if np.isfinite(p_values[i]) else np.nan,
1280         "stars": str(stars[i]),
1281     })
1282
1283     df = pd.DataFrame(rows)
1284     write_header = not os.path.exists(csv_path)
1285     df.to_csv(csv_path, mode="a", header=write_header, index=False)
1286     print(f"Saved plot data ({panel}) -> {csv_path}")
1287     return df
1288
1289
1290 # %%
1291 def make_clf():
1292     return Pipeline([
1293         ("scaler", StandardScaler()),
1294         ("clf", SVC(**svc_params))
1295     ])
1296
1297 # Fix axis order into (sub, trial, block, feat)
1298 def to_stbf(X):
1299     X = np.asarray(X)
1300
1301     shape = X.shape
1302     candidates = []
1303     for s_axis in range(4):
1304         for t_axis in range(4):
1305             for b_axis in range(4):
1306                 for f_axis in range(4):
1307                     if len({s_axis, t_axis, b_axis, f_axis}) != 4:
1308                         continue
1309                     s, t, b, f = shape[s_axis], shape[t_axis], shape[b_axis], shape[f_axis]
1310                     # basic heuristics
1311                     if abs(s - 35) <= 2 and t == 40 and b == 6 and f >= 40:
1312                         candidates.append((s_axis, t_axis, b_axis, f_axis))
1313
1314     if not candidates:
1315         # fallback: assume already correct
1316         return X
1317
1318     # take first match
1319     s_axis, t_axis, b_axis, f_axis = candidates[0]
1320     X2 = np.moveaxis(X, [s_axis, t_axis, b_axis, f_axis], [0, 1, 2, 3])
1321     return X2
1322
1323 # %% [markdown]
1324 # Load MAT and locate feature array
1325
1326 # %%
1327 mat = sio.loadmat(in_file)
1328 keys = [k for k in mat.keys() if not k.startswith("__")]
1329 print("MAT keys:", keys)
1330
1331 feat_key = "features" if "features" in keys else keys[0]
1332 X_raw = np.asarray(mat[feat_key])
1333
1334 print("Loaded key:", feat_key)
1335 print("Raw shape:", X_raw.shape, "dtype:", X_raw.dtype)
1336
1337 X_all = to_stbf(X_raw).astype(np.float32)
1338 print("Coerced shape (sub, trial, block, feat):", X_all.shape)
1339
1340 n_subjects, n_trials, n_blocks, n_features = X_all.shape

```



```

1341 print("Subjects:", n_subjects, "Trials:", n_trials, "Blocks:", n_blocks, "Features:", n_features)
1342
1343 # %%
1344 # LOSO returning per subject per block accuracy and ITR
1345 def loso_eval_subject_blockwise(X, make_clf):
1346     X = np.asarray(X)
1347     nS, nT, nB, nF = X.shape
1348
1349     y_one = np.repeat(np.arange(nT), nB)
1350
1351     acc_blk = np.zeros((nS, nB), dtype=float)
1352     itr_blk = np.zeros((nS, nB), dtype=float)
1353
1354     acc_mean = np.zeros(nS, dtype=float)
1355     acc_sd = np.zeros(nS, dtype=float)
1356     itr_mean = np.zeros(nS, dtype=float)
1357     itr_sd = np.zeros(nS, dtype=float)
1358
1359     for s in range(nS):
1360         train = [i for i in range(nS) if i != s]
1361
1362         Xtr = X[train].reshape(len(train) * nT * nB, nF)
1363         ytr = np.tile(y_one, len(train))
1364
1365         Xte = X[s].reshape(nT * nB, nF)
1366         yte = y_one
1367
1368         clf = make_clf()
1369         clf.fit(Xtr, ytr)
1370         yhat = clf.predict(Xte)
1371
1372         for bl in range(nB):
1373             sl = slice(bl * nT, (bl + 1) * nT)
1374             a = float(np.mean(yhat[sl] == yte[sl]))
1375             acc_blk[s, bl] = a
1376             itr_blk[s, bl] = itr_from_acc(a, N=nT, total_t=total_t)
1377
1378             acc_mean[s] = float(np.mean(acc_blk[s]))
1379             acc_sd[s] = float(np.std(acc_blk[s], ddof=1)) if nB > 1 else 0.0
1380             itr_mean[s] = float(np.mean(itr_blk[s]))
1381             itr_sd[s] = float(np.std(itr_blk[s], ddof=1)) if nB > 1 else 0.0
1382
1383             if (s + 1) % 5 == 0 or (s + 1) == nS:
1384                 print(f"Progress: subject {s + 1}/{nS}")
1385
1386     return acc_blk, itr_blk, acc_mean, acc_sd, itr_mean, itr_sd
1387
1388 # %%
1389 X_fbcca = X_all[:, :K]
1390 X_phase = X_all
1391
1392 print("X_fbcca:", X_fbcca.shape)
1393 print("X_phase:", X_phase.shape)
1394 print("Phase tail length:", n_features - K)
1395
1396 print("Running LOSO: FBCCA")
1397 acc_fb_blk, itr_fb_blk, acc_fb_m, acc_fb_sd, itr_fb_m, itr_fb_sd = loso_eval_subject_blockwise(
1398     X_fbcca, make_clf)
1399
1400 print("Running LOSO: FBCCA + Phase")
1401 acc_ph_blk, itr_ph_blk, acc_ph_m, acc_ph_sd, itr_ph_m, itr_ph_sd = loso_eval_subject_blockwise(

```

```

X_phase, make_clf)
1402
1403 print("Done rerun")
1404
1405 # %%
1406 # Per subject per point t tests across blocks
1407 p_acc_sub = np.array([ttest_rel(acc_ph_blk[s], acc_fb_blk[s], nan_policy="omit").pvalue for s in
    range(n_subjects)], dtype=float)
1408 p_itr_sub = np.array([ttest_rel(itr_ph_blk[s], itr_fb_blk[s], nan_policy="omit").pvalue for s in
    range(n_subjects)], dtype=float)
1409
1410 stars_acc_sub = np.array([star_from_p(p) for p in p_acc_sub], dtype=object)
1411 stars_itr_sub = np.array([star_from_p(p) for p in p_itr_sub], dtype=object)
1412
1413 print("Acc p value range:", np.nanmin(p_acc_sub), "to", np.nanmax(p_acc_sub))
1414 print("ITR p value range:", np.nanmin(p_itr_sub), "to", np.nanmax(p_itr_sub))
1415
1416 # %%
1417 # Save per subject results table
1418 df_sub = pd.DataFrame({
1419     "subject": np.arange(1, n_subjects + 1, dtype=int),
1420
1421     "acc_fbcca_mean": acc_fb_m,
1422     "acc_fbcca_sd": acc_fb_sd,
1423     "itr_fbcca_mean": itr_fb_m,
1424     "itr_fbcca_sd": itr_fb_sd,
1425
1426     "acc_phase_mean": acc_ph_m,
1427     "acc_phase_sd": acc_ph_sd,
1428     "itr_phase_mean": itr_ph_m,
1429     "itr_phase_sd": itr_ph_sd,
1430
1431     "acc_pvalue": p_acc_sub,
1432     "acc_stars": stars_acc_sub,
1433     "itr_pvalue": p_itr_sub,
1434     "itr_stars": stars_itr_sub,
1435 })
1436
1437 csv_sub = os.path.join(out_dir, "per_subject_9ch_fbcca_vs_phase.csv")
1438 df_sub.to_csv(csv_sub, index=False)
1439 print("Saved:", csv_sub)
1440
1441 # %%
1442 # Export plot CSV
1443 plot_csv = os.path.join(out_dir, "fig_9ch_subjects.csv")
1444 if os.path.exists(plot_csv):
1445     os.remove(plot_csv)
1446
1447 subjects = np.arange(1, n_subjects + 1, dtype=int)
1448
1449 save_plot_data_csv(
1450     csv_path=plot_csv,
1451     plot_id="9ch_subjects",
1452     panel="acc",
1453     x_values=subjects,
1454     series_dict={
1455         "fbcca": (acc_fb_m * 100, acc_fb_sd * 100),
1456         "phase": (acc_ph_m * 100, acc_ph_sd * 100),
1457     },
1458     unit="percent",
1459     p_values=p_acc_sub,
1460

```

```

1461     stars=stars_acc_sub
1462 )
1463
1464 save_plot_data_csv(
1465     csv_path=plot_csv,
1466     plot_id="9ch_subjects",
1467     panel="itr",
1468     x_values=subjects,
1469     series_dict={
1470         "fbcca": (itr_fb_m, itr_fb_sd),
1471         "phase": (itr_ph_m, itr_ph_sd),
1472     },
1473     unit="bits_per_min",
1474     p_values=p_itr_sub,
1475     stars=stars_itr_sub
1476 )
1477
1478 print("Saved:", plot_csv)
1479
1480
1481 # %% [markdown]
1482 # ##### Plot: subjects
1483
1484 # %%
1485
1486 fig, ax = plt.subplots(1, 2, figsize=(FIG_W, FIG_H), constrained_layout=True)
1487
1488 ax0 = ax[0]
1489 ax0.errorbar(subjects, acc_fb_m * 100, yerr=acc_fb_sd * 100, fmt="o", color='blue', capsize=4,
1490             label="FBCCA")
1491 ax0.errorbar(subjects, acc_ph_m * 100, yerr=acc_ph_sd * 100, fmt="o", color='red', capsize=4,
1492             label="FBCCA + Phase")
1493 ax0.set_xlabel("Subject")
1494 ax0.set_ylabel("Accuracy (%)")
1495 ax0.set_ylim(-10, 130)
1496 ax0.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
1497
1498 y_span = ax0.get_ylim()[1] - ax0.get_ylim()[0]
1499 y_top = ax0.get_ylim()[1] - 0.01 * y_span
1500 for i, s in enumerate(subjects):
1501     draw_vertical_stars(ax0, s, float(p_acc_sub[i]), y_top, y_span)
1502
1503 ax1 = ax[1]
1504 ax1.errorbar(subjects, itr_fb_m, yerr=itr_fb_sd, fmt="o", color='blue', capsize=4, label="FBCCA")
1505 ax1.errorbar(subjects, itr_ph_m, yerr=itr_ph_sd, fmt="o", color='red', capsize=4, label="FBCCA +
1506             Phase")
1507 ax1.set_xlabel("Subject")
1508 ax1.set_ylabel("ITR (bits/min)")
1509 ax1.set_ylim(-10, 170)
1510 ax1.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
1511
1512 y_span = ax1.get_ylim()[1] - ax1.get_ylim()[0]
1513 y_top = ax1.get_ylim()[1] - 0.01 * y_span
1514 for i, s in enumerate(subjects):
1515     draw_vertical_stars(ax1, s, float(p_itr_sub[i]), y_top, y_span)
1516
1517 pdf_out = os.path.join(plots_dir, "Results_subjects_9_channels.pdf")
1518 plt.savefig(pdf_out)
1519 plt.show()
1520 print("Saved:", pdf_out)
1521
1522
1523

```

```

1520 # %% [markdown]
1521 # ## Time Window Grid Analysis
1522
1523 # %%
1524 grid_dir = r"\TW_Grid"
1525 out_dir = grid_dir
1526
1527 svc_params = dict(kernel="rbf", C=1, gamma=0.01)
1528 K = 40
1529 setups = ["All9", "Oz"]
1530
1531
1532 # Helpers
1533 def _parse_tw(fname):
1534     base = os.path.splitext(os.path.basename(fname))[0]
1535     m = re.search(r"_tw(?:P<tw>\d+p\d+)$", base)
1536     if not m:
1537         return None
1538     return float(m.group("tw").replace("p", "."))
1539
1540 def _parse_setup_from_fname(fname):
1541     base = os.path.splitext(os.path.basename(fname))[0]
1542     m = re.search(r"FBCCA_PHASE_(?:P<setup>.+?)_tw\d+p\d+$", base)
1543     if not m:
1544         return None
1545     return m.group("setup")
1546
1547 def loso_eval_subjectwise(X, svc_params, t_w, t_break, t_latency, t_comp):
1548     n_subjects, n_trials_local, n_blocks_local, n_feats = X.shape
1549     if (n_trials_local != n_trials) or (n_blocks_local != n_blocks):
1550         raise ValueError(f"Unexpected shape {X.shape}")
1551
1552     total_t = float(t_w) + float(t_break) + float(t_latency) + float(t_comp)
1553     y_one = np.repeat(np.arange(n_trials), n_blocks)
1554
1555     acc = np.zeros(n_subjects, dtype=float)
1556     itr = np.zeros(n_subjects, dtype=float)
1557
1558     pipe = Pipeline([
1559         ("scaler", StandardScaler()),
1560         ("svc", SVC(**svc_params)),
1561     ])
1562
1563     for s in range(n_subjects):
1564         train = [i for i in range(n_subjects) if i != s]
1565
1566         Xtr = X[train].reshape(len(train) * n_trials * n_blocks, n_feats)
1567         ytr = np.tile(y_one, len(train))
1568
1569         Xte = X[s].reshape(n_trials * n_blocks, n_feats)
1570         yte = y_one
1571
1572         pipe.fit(Xtr, ytr)
1573         yhat = pipe.predict(Xte)
1574
1575         a = float(np.mean(yhat == yte))
1576         acc[s] = a
1577         itr[s] = itr_from_acc(a, N=n_trials, total_t=total_t)
1578
1579     return acc, itr
1580
1581 # Discover files

```

```

1582 all_files = sorted(glob.glob(os.path.join(grid_dir, "FBCCA_PHASE*_tw*.mat")))
1583 if not all_files:
1584     raise FileNotFoundError(f"No grid MAT files found in {grid_dir}")
1585
1586 print(f"Found {len(all_files)} grid files")
1587 print("Setups to analyze:", setups)
1588
1589 # Run analysis per setup
1590 t_global_start = time.time()
1591
1592 for setup in setups:
1593     print("Starting setup:", setup)
1594
1595     setup_files = [f for f in all_files if _parse_setup_from_fname(f) == setup]
1596     if not setup_files:
1597         print("No files found for setup:", setup)
1598         continue
1599
1600     setup_files = sorted(setup_files, key=_parse_tw)
1601     print(f"{len(setup_files)} time windows detected")
1602
1603     rows = []
1604     long_rows = []
1605
1606     t_setup_start = time.time()
1607
1608     for i, mp in enumerate(setup_files, 1):
1609         t_w = _parse_tw(mp)
1610         if t_w is None:
1611             continue
1612
1613         print(f"\n[{setup}] t_w = {t_w:.2f} s  ({i}/{len(setup_files)})")
1614
1615         d = sio.loadmat(mp)
1616         X = np.asarray(d["features"]).astype(np.float32)
1617
1618         t_break = float(np.asarray(d["t_break"]).squeeze())
1619         t_latency = float(np.asarray(d["t_latency"]).squeeze())
1620         t_comp = float(np.asarray(d["t_comp"]).squeeze())
1621
1622         # Split features into FBCCA and FBCCA + Phase
1623         X_fb = X[:, :, :K]
1624         X_ph = X
1625
1626         print(" Running LOSO: FBCCA")
1627         acc_fb_sub, itr_fb_sub = loso_eval_subjectwise(
1628             X_fb, svc_params, t_w, t_break, t_latency, t_comp
1629         )
1630
1631         print(" Running LOSO: FBCCA + Phase")
1632         acc_ph_sub, itr_ph_sub = loso_eval_subjectwise(
1633             X_ph, svc_params, t_w, t_break, t_latency, t_comp
1634         )
1635
1636         # Paired tests
1637         p_acc = float(ttest_rel(acc_ph_sub, acc_fb_sub, nan_policy="omit").pvalue)
1638         p_itr = float(ttest_rel(itr_ph_sub, itr_fb_sub, nan_policy="omit").pvalue)
1639
1640         print(
1641             f" Acc: FBCCA {acc_fb_sub.mean():.3f}, Phase {acc_ph_sub.mean():.3f}, p={p_acc:.3g}"
1642         )
1643         print(

```

```

1644         f" ITR: FBCCA {itr_fb_sub.mean():.2f}, Phase {itr_ph_sub.mean():.2f}, p={p_itr:.3g}"
1645     )
1646
1647     rows.append({
1648         "setup": setup,
1649         "t_w": float(t_w),
1650         "t_break": t_break,
1651         "t_latency": t_latency,
1652         "t_comp": t_comp,
1653
1654         "acc_fb_mean": float(np.mean(acc_fb_sub)),
1655         "acc_fb_sd": float(np.std(acc_fb_sub, ddof=1)),
1656         "itr_fb_mean": float(np.mean(itr_fb_sub)),
1657         "itr_fb_sd": float(np.std(itr_fb_sub, ddof=1)),
1658
1659         "acc_ph_mean": float(np.mean(acc_ph_sub)),
1660         "acc_ph_sd": float(np.std(acc_ph_sub, ddof=1)),
1661         "itr_ph_mean": float(np.mean(itr_ph_sub)),
1662         "itr_ph_sd": float(np.std(itr_ph_sub, ddof=1)),
1663
1664         "acc_pvalue": p_acc,
1665         "itr_pvalue": p_itr,
1666
1667         "mat_file": mp,
1668     })
1669
1670     for s in range(len(acc_fb_sub)):
1671         long_rows.append({
1672             "setup": setup,
1673             "t_w": float(t_w),
1674             "subject": int(s + 1),
1675             "acc_fb": float(acc_fb_sub[s]),
1676             "itr_fb": float(itr_fb_sub[s]),
1677             "acc_ph": float(acc_ph_sub[s]),
1678             "itr_ph": float(itr_ph_sub[s]),
1679             "acc_diff": float(acc_ph_sub[s] - acc_fb_sub[s]),
1680             "itr_diff": float(itr_ph_sub[s] - itr_fb_sub[s]),
1681         })
1682
1683     # Save outputs for this setup
1684     res_df = pd.DataFrame(rows).sort_values("t_w").reset_index(drop=True)
1685     long_df = pd.DataFrame(long_rows).sort_values(["t_w", "subject"]).reset_index(drop=True)
1686
1687     res_csv = os.path.join(out_dir, f"tw_grid_fbcca_vs_phase_{setup}.csv")
1688     long_csv = os.path.join(out_dir, f"tw_grid_fbcca_vs_phase_{setup}_per_subject.csv")
1689
1690     res_df.to_csv(res_csv, index=False)
1691     long_df.to_csv(long_csv, index=False)
1692
1693     t_setup_end = time.time()
1694     print(f"\nFinished setup {setup} in {(t_setup_end - t_setup_start)/60:.1f} minutes")
1695     print("Saved:", res_csv)
1696     print("Saved:", long_csv)
1697
1698     t_global_end = time.time()
1699     print(f"All setups finished in {(t_global_end - t_global_start)/60:.1f} minutes")
1700
1701
1702     # %%
1703     # Plotting TW gridsearches
1704     out_dir = r"\TW_Grid"
1705     setups = ["A119", "Oz"]

```

```

1706
1707 for setup in setups:
1708     csv_in = os.path.join(out_dir, f"tw_grid_fbcca_vs_phase_{setup}.csv")
1709     if not os.path.exists(csv_in):
1710         print("Missing:", csv_in)
1711         continue
1712
1713     df = pd.read_csv(csv_in).sort_values("t_w").reset_index(drop=True)
1714
1715     tw = df["t_w"].values
1716
1717     # accuracy in percent for plotting
1718     acc_fb_m = df["acc_fb_mean"].values * 100
1719     acc_fb_sd = df["acc_fb_sd"].values * 100
1720     acc_ph_m = df["acc_ph_mean"].values * 100
1721     acc_ph_sd = df["acc_ph_sd"].values * 100
1722
1723     itr_fb_m = df["itr_fb_mean"].values
1724     itr_fb_sd = df["itr_fb_sd"].values
1725     itr_ph_m = df["itr_ph_mean"].values
1726     itr_ph_sd = df["itr_ph_sd"].values
1727
1728     acc_p = df["acc_pvalue"].values
1729     itr_p = df["itr_pvalue"].values
1730
1731     acc_stars = [star_from_p(float(p)) for p in acc_p]
1732     itr_stars = [star_from_p(float(p)) for p in itr_p]
1733
1734     fig, ax = plt.subplots(1, 2, figsize=(FIG_W, FIG_H), constrained_layout=True)
1735
1736     # Accuracy
1737     ax0 = ax[0]
1738     ax0.errorbar(tw, acc_fb_m, yerr=acc_fb_sd, fmt="o", color='blue', capsize=4, label="FBCCA")
1739     ax0.errorbar(tw, acc_ph_m, yerr=acc_ph_sd, fmt="o", color='red', capsize=4, label="FBCCA +
1740     Phase")
1741     ax0.set_xlabel("Time (s)")
1742     ax0.set_ylabel("Accuracy (%)")
1743     ax0.set_ylim(-10, 130)
1744     ax0.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
1745
1746     y_span = ax0.get_ylim()[1] - ax0.get_ylim()[0]
1747     y_top = ax0.get_ylim()[1] - 0.01 * y_span
1748     for i, tw_i in enumerate(tw):
1749         if acc_stars[i]:
1750             draw_vertical_stars(ax0, tw_i, float(acc_p[i]), y_top, y_span)
1751
1752     # ITR
1753     ax1 = ax[1]
1754     ax1.errorbar(tw, itr_fb_m, yerr=itr_fb_sd, fmt="o", color='blue', capsize=4, label="FBCCA")
1755     ax1.errorbar(tw, itr_ph_m, yerr=itr_ph_sd, fmt="o", color='red', capsize=4, label="FBCCA +
1756     Phase")
1757     ax1.set_xlabel("Time (s)")
1758     ax1.set_ylabel("ITR (bits/min)")
1759     ax1.set_ylim(-10, 170)
1760     ax1.legend(bbox_to_anchor=(0, -0.3), loc='lower left', frameon=False)
1761
1762     y_span = ax1.get_ylim()[1] - ax1.get_ylim()[0]
1763     y_top = ax1.get_ylim()[1] - 0.01 * y_span
1764     for i, tw_i in enumerate(tw):
1765         if itr_stars[i]:
1766             draw_vertical_stars(ax1, tw_i, float(itr_p[i]), y_top, y_span)

```

```
1766     # Save figure
1767     pdf_out = os.path.join(out_dir, f"Results_tw_grid_{setup}.pdf")
1768     plt.savefig(pdf_out)
1769     plt.show()
1770     print("Saved:", pdf_out)
1771
1772     # Save plot data used
1773     plot_csv = os.path.join(out_dir, f"fig_tw_grid_{setup}.csv")
1774
1775     save_plot_data_csv(
1776         csv_path=plot_csv,
1777         plot_id=f"tw_grid_{setup}",
1778         panel="acc",
1779         x_values=[f"{v:.2f}" for v in tw],
1780         series_dict={
1781             "fbcca": (acc_fb_m, acc_fb_sd),
1782             "phase": (acc_ph_m, acc_ph_sd),
1783         },
1784         unit="percent",
1785         p_values=acc_p,
1786         stars=acc_stars,
1787     )
1788
1789     save_plot_data_csv(
1790         csv_path=plot_csv,
1791         plot_id=f"tw_grid_{setup}",
1792         panel="itr",
1793         x_values=[f"{v:.2f}" for v in tw],
1794         series_dict={
1795             "fbcca": (itr_fb_m, itr_fb_sd),
1796             "phase": (itr_ph_m, itr_ph_sd),
1797         },
1798         unit="bits_per_min",
1799         p_values=itr_p,
1800         stars=itr_stars,
1801     )
```